

Contents

SDLC Fundamentals for Managers: Individual Hands-on Lab Guide	3
1: The Modern Project Manager's Role in the Cycle	3
2: Waterfall vs. Agile vs. Hybrid Delivery Models	5
3: The Shift-Left Mentality in Risk Management and QA	7
4: Overview of Standard SDLC Phases and Stage Gates	8
CMMI Fundamentals for Project Managers: Individual Hands-on Lab Guide	11
1: Introduction to Capability Maturity Model Integration (CMMI).....	11
2: Understanding Maturity Levels (1–5) and What They Mean for Project Governance	13
3: Mapping CMMI to SDLC Phases and Stage Gates	15
4: How CMMI Strengthens Planning, Estimation, Risk Management, and Stakeholder Alignment	16
5: Integrating CMMI with Agile, Scrum, and Hybrid Delivery Models.....	18
Agile & Scrum for Project Managers: Individual Hands-on Lab Guide.....	20
1: Navigating Sprints, Daily Stand-Ups, and Retrospectives	20
2: Translating Business Requirements into Epics and User Stories	21
3: Velocity Tracking, Story Pointing, and Capacity Planning.....	23
Requirement Gathering and Analysis: Individual Hands-on Lab Guide.....	25
1: Collaborating with Stakeholders to Identify Edge Cases Early	25
2: Defining the Minimum Viable Product (MVP) and Release Phases	27
Risk Management & Technical Debt: Individual Hands-on Lab Guide	29
1: Identifying and Prioritizing Project and Technical Risks	29
2: Balancing Feature Delivery with Technical Debt Repayment	31
Cross-Functional Team Collaboration: Individual Hands-on Lab Guide	33
1: Aligning Engineering, Design, and QA Resources.....	33
2: Facilitating Effective Communication Channels.....	35
3: Fostering a Culture of Accountability and Psychological Safety	38
4: Managing Stakeholder Expectations and Feedback Loops	40
Release Planning and Delivery (CI/CD Overview): Individual Hands-on Lab Guide.....	42
1: Understanding Continuous Integration and Deployment Flows	42
2: Managing Build Artifacts, Versions, and Release Branches	44

3: Coordinating Blue-Green, Beta, and Canary Deployments.....	46
Quality Assurance & UAT: Individual Hands-on Lab Guide	48
1: Integrating QA Early to Reduce Defect Resolution Costs	48
2: Managing the User Acceptance Testing (UAT) Phase	50
3: Tracking Bugs, Severities, and Triage Processes	52
4: Establishing Clear Acceptance Criteria and Definition of Done (DoD)	54
Metrics, Reporting, and Dashboards: Individual Hands-on Lab Guide	56
1: Configuring Actionable Project Dashboards in Jira	56
2: Analyzing Burndown, Burnup, and Cumulative Flow Diagrams	58
Change Management and Scope Creep: Individual Hands-on Lab Guide.....	60
1: Implementing Formal Change Request and Approval Processes	60
2: Negotiating Scope Alterations Against Time and Budget Constraints	62
3: Maintaining Alignment with Core Business Objectives.....	64
CMMI in Delivery, Quality, and Governance: Individual Hands-on Lab Guide.....	66
1: Using CMMI KPAs to Strengthen QA, UAT, and Release Management	66
2: Establishing Measurable KPIs Aligned with Maturity Goals	68
3: Ensuring Consistent Acceptance Criteria and Definition of Done (DoD)	69
4: How CMMI Supports Continuous Improvement and Post-Release Learning Cycles	71
5: Preparing Teams for CMMI Appraisals (SCAMPI A/B/C)	72
Post-Release Maintenance and Incident Response: Individual Hands-on Lab Guide	74
1: Structuring Hotfixes and Emergency Maintenance Releases	74
2: Conducting Post-Incident Reviews (PIR) and Root Cause Analysis	76

SDLC Fundamentals for Managers: Individual Hands-on Lab Guide

1: The Modern Project Manager's Role in the Cycle

Activity 1.1: Building the Modern PM Governance Matrix in Confluence

- **Tool:** Confluence
- **Objective:** Transition your operational profile from a traditional administrative coordinator to a modern servant leader and CMMI process enabler by establishing an institutional Team Working Agreement and Governance Charter.

Step-by-Step Instructions:

1. Open your Confluence workspace and navigate to your assigned project space.
2. Click **Create** (+) in the top navigation bar to open a blank page.
3. In the page title field, enter: [Your Name] - Modern PM Governance Charter & Team Working Agreement.
4. Type /table and press **Enter** to insert a 4-column, 5-row table.
5. Populate the header row with the following column titles:
 - Lifecycle Phase
 - Traditional Administrative PM Habit
 - Modern PM Enabler / Value Facilitator Role
 - CMMI Practice Area Alignment
6. Fill in the rows with the following operational behaviors:
 - **Row 1:**
 - *Phase:* Backlog Refinement & Estimation
 - *Traditional Habit:* Assigning hours to engineers and demanding fixed delivery dates.
 - *Modern PM Role:* Facilitating story sizing sessions, removing ambiguity, and tracking velocity trends.
 - *CMMI Alignment:* Project Planning (PP) & Requirements Management (REQM)
 - **Row 2:**
 - *Phase:* Daily Execution & Flow
 - *Traditional Habit:* Running 45-minute daily meetings asking for individual status updates.
 - *Modern PM Role:* Monitoring blocker flags on the board, eliminating external dependencies, and protecting team focus.
 - *CMMI Alignment:* Project Monitoring and Control (PMC)
 - **Row 3:**
 - *Phase:* Quality & Verification
 - *Traditional Habit:* Pushing testing to end of cycle to hit schedule deadlines.
 - *Modern PM Role:* Enforcing Definition of Done (DoD) criteria and reviewing test automation pass rates before deployment.
 - *CMMI Alignment:* Verification (VER) & Process and Product Quality Assurance (PPQA)

- **Row 4:**
 - *Phase:* Sprint Retrospective
 - *Traditional Habit:* Writing a subjective status summary for management and shelving action items.
 - *Modern PM Role:* Facilitating blameless root-cause analysis and logging continuous improvements directly into the next sprint backlog.
 - *CMMI Alignment:* Causal Analysis and Resolution (CAR)
- 7. Below the table, type /panel and press **Enter** to insert an **Info Panel**.
- 8. Set the panel icon to a checkmark and add the heading: Executive Escalation & SLA Protocol.
- 9. Inside the panel, draft a 3-bullet protocol specifying:
 - Any impediment unresolved after **4 hours** is escalated to the modern PM.
 - Any technical dependency between squads blocked for **24 hours** triggers a PM-led cross-team sync.
 - Scope alterations exceeding **10% of sprint capacity** require a formal trade-off review with the sponsor.
- 10. Click **Publish** in the top right corner.

Activity 1.2: Configuring Modern PM Blocker & Risk Tracking in Jira

- **Tool:** Jira Software
- **Objective:** Configure a Jira Scrum board to visually identify flow bottlenecks and automate blocker notifications, enabling proactive servant leadership without intrusive micromanagement.

Step-by-Step Instructions:

1. Open Jira Software and navigate to your assigned project board.
2. Click the **Project Settings** icon (gear) in the bottom-left sidebar.
3. Select **Issue Types** from the left navigation panel, then click **Add issue type**.
4. Set the name to Blocker / Impediment, select an alert icon (or Bug/Warning icon), set the type to **Standard Issue Type**, and click **Create**.
5. Return to your board view, click the **three dots (...)** in the upper right corner, and select **Board settings**.
6. Select the **Card colors** tab from the settings menu.
7. Set the **Color based on** dropdown to Queries.
8. In the JQL Query field, enter: `issueType = "Blocker / Impediment" OR flag = "Impediment"`, select the color **Red**, and click **Add**.
9. Click the **Card layout** tab. Under the **Active sprints** section, select Days in column and click **Add** (this exposes work stagnation visually).
10. Click the **Back to board** link in the top left corner.
11. Click **+ Create** in the top navigation bar to generate a test scenario:
 - *Issue Type:* User Story
 - *Summary:* Implement OAuth2.0 Token Validation Engine
 - *Story Points:* 5
12. Click into the newly created card on your board, click the **Actions (...)** menu on the issue details pane, and select **Add flag**.
13. Observe the card turn yellow/highlighted with an impediment indicator.

14. Click + **Create** again:
 - *Issue Type*: Blocker / Impediment
 - *Summary*: Production Database Gateway Latency Exceeds 800ms SLA
 - *Linked Issues*: Select blocks, then search and select the OAuth story created in Step 11.
15. Verify that the blocker ticket renders with the red visual card highlight on your active board.

2: Waterfall vs. Agile vs. Hybrid Delivery Models

Activity 2.1: Designing a Hybrid SDLC Delivery Pipeline in Lucidchart

- **Tool**: Lucidchart (or Miro)
- **Objective**: Design an enterprise Hybrid SDLC architecture diagram that layers iterative Agile two-week sprints inside a CMMI-compliant, milestone-governed project delivery framework.

Step-by-Step Instructions:

1. Log in to Lucidchart and click + **New > Lucidchart > Blank Document**.
2. Rename the document in the top-left corner: [Your Name] - Enterprise Hybrid Delivery Blueprint.
3. From the left shape library, drag a **Horizontal Container (Swimlane)** onto the canvas. Set it to **3 swimlanes**:
 - Label Swimlane 1: Phase Governance & Stage Gates (CMMI Level 3)
 - Label Swimlane 2: Agile Delivery Engine (Scrum Sprints)
 - Label Swimlane 3: Continuous Verification & Integration (CI/CD Pipeline)
4. In **Swimlane 1**, drag 4 diamond shapes (Decision points) horizontally across the lane:
 - Label Diamond 1 (Start): Gate 1: Business Case & Architecture Approval
 - Label Diamond 2 (Mid-project): Gate 2: Mid-Release Baseline Audit
 - Label Diamond 3 (Pre-release): Gate 3: Security & Operational Readiness Review
 - Label Diamond 4 (End): Gate 4: Production Go/No-Go Sign-Off
5. In **Swimlane 2**, directly between Gate 1 and Gate 3, place 3 looped sequence groups representing sprints:
 - Block A: Sprint 1-2: Core Platform & Data Schema
 - Block B: Sprint 3-4: Feature APIs & Third-Party Integrations
 - Block C: Sprint 5-6: UI/UX & End-to-End User Journeys
6. In **Swimlane 3**, place 3 rounded process rectangles running parallel below the sprint blocks:
 - Block A: Automated Unit Testing & Linting (>80% Code Coverage)
 - Block B: Static Security Scans & Integration Smoke Tests
 - Block C: Automated Regression Suite & Staging Deployment
7. Draw connector lines showing governance and feedback flows:
 - Connect **Gate 1** to **Sprint 1-2**.
 - Draw bidirectional vertical dashed arrows between each **Sprint Block** and its corresponding **Verification Block** in Swimlane 3.
 - Connect the completion of **Sprint 5-6** directly to **Gate 3**.
 - Connect **Gate 3** to **Gate 4**.
8. Apply color coding:
 - Fill Gate diamonds with light purple.
 - Fill Sprint boxes with light blue.
 - Fill CI/CD verification boxes with light green.

9. Click **File > Export > PNG** (or keep the direct link) for submission.

Activity 2.2: Slicing a Waterfall Scope into an Agile Release Plan in Jira

- **Tool:** Jira Software
- **Objective:** Deconstruct a rigid, fixed-scope statement of work (SOW) into an Agile release structure with iterative Fix Versions and traceable user stories.

Step-by-Step Instructions:

1. Navigate to your project in Jira Software.
2. In the left project menu, click on **Releases**.
3. Click **Create version** in the top right corner.
4. Set the following fields:
 - *Name:* Release 1.0 - Core Banking API (Hybrid Baseline)
 - *Start date:* Select today's date.
 - *Release date:* Select a date 6 weeks from today.
 - *Description:* Foundational transaction engine and ledger integration satisfying CMMI requirements baseline.
5. Click **Save**.
6. Click **Create version** again:
 - *Name:* Release 2.0 - Customer Mobile Portal
 - *Start date:* Select the day after Release 1.0 ends.
 - *Release date:* Select a date 6 weeks following Release 1.0.
 - *Description:* Public UI, notification services, and self-service onboarding.
7. Click **Save**.
8. Navigate to **Backlog** in the left sidebar.
9. Click the **Versions** panel toggle (or expand the **Versions** drop-down filter on the backlog).
10. Click **+ Create issue** at the bottom of the backlog to add the following 3 items:
 - Item 1: [Story] As an API client, I need a secure endpoint to post debit transactions
 - Item 2: [Story] As a compliance officer, I need transactions logged to an immutable audit trail
 - Item 3: [Story] As a mobile user, I want biometric login to view my account balance
11. Drag and drop **Item 1** and **Item 2** into Release 1.0 - Core Banking API (Hybrid Baseline).
12. Drag and drop **Item 3** into Release 2.0 - Customer Mobile Portal.
13. Click into **Item 1**, scroll down the right-hand details pane, locate the **Fix versions** field, and confirm it shows Release 1.0 - Core Banking API (Hybrid Baseline).

3: The Shift-Left Mentality in Risk Management and QA

Activity 3.1: Building a Shift-Left Risk Breakdown Structure (RBS) in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Map delivery and architectural risks to the earliest possible SDLC entry points, establishing mitigation mechanisms before code is written.

Step-by-Step Instructions:

1. Open Miro and click **+ New board > Create a blank board**.
2. Name the board: [Your Name] - Shift-Left Risk Breakdown & Prevention Framework.
3. Add a title at the top of the canvas: Shift-Left Risk & QA Architecture.
4. Create 4 vertical columns using the **Rectangle** tool (set fill to transparent with a solid border):
 - Column 1: Requirements & Discovery Phase
 - Column 2: Architecture & Sprint Planning Phase
 - Column 3: Development & Peer Review Phase
 - Column 4: Traditional Downstream Phase (Late Shift-Right Anti-Pattern)
5. Color Column 4 light red to designate it as the high-risk, expensive anti-pattern zone.
6. In Column 4, add two sticky notes describing downstream failures:
 - Sticky 1: Security vulnerability discovered 2 days before production deployment.
 - Sticky 2: Ambiguous regulatory requirement causes 40% rework during final UAT.
7. Shift these risks left into Columns 1, 2, and 3 by placing the following sticky notes:
 - **In Column 1 (Requirements):**
 - Sticky (Yellow): Risk: Regulatory requirement mismatch.
 - Sticky (Green): Shift-Left Action: Integrate Compliance SME directly into Three Amigos refinement before story is approved.
 - **In Column 2 (Planning):**
 - Sticky (Yellow): Risk: Architectural bottleneck under peak concurrency.
 - Sticky (Green): Shift-Left Action: Run a 1-sprint timeboxed Architectural Spike to benchmark Redis vs. PostgreSQL before committing to features.
 - **In Column 3 (Development):**
 - Sticky (Yellow): Risk: Insecure dependencies and injection vulnerabilities.
 - Sticky (Green): Shift-Left Action: Embed automated static security analysis (SAST) directly into pre-commit Git hooks.
8. Draw directional connection arrows from the green preventive actions in Columns 1–3 pointing toward the red failures in Column 4, adding the text label Eliminates Defect on the arrow lines.
9. Add an arrow along the bottom across all columns from left to right with the label: Cost to Fix Increases 10x - 100x ->.

Activity 3.2: Implementing Automated Quality Stage Gates in Jira Workflows

- **Tool:** Jira Software
- **Objective:** Configure an enforcement mechanism in a Jira project workflow that stops user stories from advancing to development without clear, testable acceptance criteria (Shift-Left QA).

Step-by-Step Instructions:

1. In Jira, navigate to your project and click **Project settings** (gear icon) in the sidebar.
2. Click **Workflows** in the left sidebar menu.
3. Locate the workflow assigned to **Story** and click the **Pencil (Edit)** icon to open the workflow editor.
4. Toggle to the **Diagram** view in the workflow editor.
5. Identify the transition arrow connecting Backlog (or To Do) to In Progress (or In Development).
6. Click directly on that transition arrow to select it.
7. In the configuration panel on the right, click **Validators**.
8. Click **Add validator**.
9. Select **Field Required Validator** and click **Add**.
10. In the **Fields required for this transition** dropdown, select Description (or custom field Acceptance Criteria if configured in your instance).
11. In the **Error message** field, type: Shift-Left Quality Rule: User Stories cannot transition to In Progress without complete Acceptance Criteria (Given-When-Then format).
12. Click **Add**.
13. Click **Publish Draft** in the yellow top banner. Select **No** to saving a backup copy, and click **Publish**.
14. Test the validation gate:
 - Go back to your board and click **+ Create**.
 - Create a test story with a Summary: Test Shift-Left Gate, but **leave the Description field completely blank**.
 - Locate the ticket in your backlog/board and attempt to drag it directly to In Progress.
 - Verify that Jira blocks the transition and displays your configured error message.

4: Overview of Standard SDLC Phases and Stage Gates

Activity 4.1: Building a CMMI-Aligned SDLC Phase-Gate Matrix in Confluence

- **Tool:** Confluence
- **Objective:** Author an institutional Stage-Gate policy that defines mandatory entry criteria, verification artifacts, and exit conditions for each SDLC phase.

Step-by-Step Instructions:

1. Open your Confluence space and click **Create (+)**.
2. Set the page title to: [Your Name] - SDLC Stage-Gate Governance Standard (CMMI Level 3).
3. Type /table and press **Enter** to insert a 5-column, 6-row table.
4. Set the column headers to:
 - SDLC Phase
 - Stage Gate Name
 - Entry Criteria
 - Mandatory Verification Artifact

- Exit / Approval Authority
- 5. Populate the rows with the following operational data:
 - **Row 1:**
 - *Phase:* Inception / Concept
 - *Gate:* Gate 0: Charter Approval
 - *Entry:* Initial business case and executive problem statement submitted.
 - *Artifact:* Project Charter, High-Level ROI Model, Initial Risk Register.
 - *Authority:* Project Sponsor & IT Steering Committee
 - **Row 2:**
 - *Phase:* Requirements Analysis
 - *Gate:* Gate 1: Definition of Ready (DoR)
 - *Entry:* Approved Charter and designated Product Owner assigned.
 - *Artifact:* Epics deconstructed into User Stories with Given-When-Then Acceptance Criteria; Traceability IDs assigned.
 - *Authority:* Product Owner & Modern PM
 - **Row 3:**
 - *Phase:* Architecture & Design
 - *Gate:* Gate 2: Architecture Baseline Review
 - *Entry:* User stories prioritized in product backlog.
 - *Artifact:* System Architecture Document (SAD), Data Flow Diagram, Security Threat Model.
 - *Authority:* Enterprise Architect & CISO Representative
 - **Row 4:**
 - *Phase:* Construction & Sprinting
 - *Gate:* Gate 3: Definition of Done (DoD)
 - *Entry:* User stories pulled into an active sprint.
 - *Artifact:* Merged Pull Request with ≥ 2 approvals, SonarQube pass (0 Blocker bugs, $\geq 80\%$ unit test coverage).
 - *Authority:* Tech Lead & QA Lead
 - **Row 5:**
 - *Phase:* Verification & Validation
 - *Gate:* Gate 4: Operational Readiness Review (ORR)
 - *Entry:* All sprint deliverables deployed to staging.
 - *Artifact:* 100% UAT business scenarios executed; zero open Sev 1/Sev 2 defects; Rollback Plan documented.
 - *Authority:* IT Director & Modern PM
- 6. Below the table, type /expand and press **Enter** to insert an expandable container.
- 7. Set the label to: Stage-Gate Exception and Waiver Process.
- 8. Inside the expand block, type: If an operational deadline forces a gate bypass, an Emergency Waiver must be logged as a CMMI Technical Debt item in Jira with an executive sign-off and an explicit remediation deadline not to exceed two subsequent sprints.
- 9. Click **Publish** in the top right corner.

Activity 4.2: Setting Up Stage-Gate Approvals and Audit Logging in Jira Service Management / Software

- **Tool:** Jira Software
- **Objective:** Configure an auditable release stage-gate review mechanism directly within Jira to provide verified compliance trails for CMMI Level 3 appraisals.

Step-by-Step Instructions:

1. In Jira, navigate to your project and open **Project Settings > Issue Types**.
2. Click **Add issue type**, enter Stage Gate Review, select a clipboard or document icon, and click **Create**.
3. Navigate to **Project Settings > Screens**.
4. Locate the screen scheme for Stage Gate Review and ensure the following fields are visible:
 - Summary
 - Description
 - Fix versions
 - Approvers (or Assignee)
 - Resolution
5. Navigate to your project board or backlog, click **+ Create** at the top, and input the following details:
 - *Issue Type:* Stage Gate Review
 - *Summary:* Gate 4 Review: Production Release Candidate v1.0.0
 - *Fix Version:* Release 1.0 - Core Banking API (Hybrid Baseline)
 - *Description:* Copy and paste this verification checklist:

Mandatory Stage-Gate Evidence Checklist:

- [] 1. All stories in release mapped to Jira Epics (REQM Compliance)
- [] 2. Static code analysis report attached: 0 Sev 1 / Blocker defects (VER Compliance)
- [] 3. Business Sponsor written UAT Sign-Off attached (VAL Compliance)
- [] 4. Rollback and runbook documented in Confluence (CM Compliance)

6. Click **Create**.
7. Open the newly created Stage Gate Review issue ticket.
8. Under the **Description**, tick the checkboxes for items 1, 2, and 3 to simulate completed verifications.
9. Leave item 4 unchecked. Add a comment: Blocked: Rollback Runbook pending final infrastructure review by DevOps lead.
10. Click **Save** on the comment.
11. Observe how the ticket maintains an auditable timestamp of who verified each check and when the gate was held, fulfilling CMMI audit requirements without paper documentation.

CMMI Fundamentals for Project Managers: Individual Hands-on Lab Guide

1: Introduction to Capability Maturity Model Integration (CMMI)

Activity 1.1: Establishing an Institutional Process Asset Library (PAL) in Confluence

- **Tool:** Confluence
- **Objective:** Establish an organization-wide Process Asset Library (PAL) repository structure that centralizes governance assets, lifecycle standards, and tailoring guidelines to fulfill core CMMI foundational expectations.

Step-by-Step Instructions:

1. Open Confluence and navigate to your assigned space.
2. Click **Create** (+) in the top navigation bar to create a root landing page.
3. In the page title field, enter: [Your Name] - Organizational Process Asset Library (PAL).
4. Type /table and press **Enter** to insert a 4-column, 5-row index table.
5. Populate the header row with the following column titles:
 - Asset Identifier
 - Process Domain / Category
 - Standard Asset Description
 - Mandatory vs. Tailorable
6. Enter the following rows into the table:
 - **Row 1:**
 - *Asset ID:* PAL-STD-001
 - *Domain:* Project Planning (PP)
 - *Description:* Sprint Planning, Capacity Allocation & Estimation Standard
 - *Category:* Tailorable (Teams may choose Fibonacci story points or ideal engineering hours)
 - **Row 2:**
 - *Asset ID:* PAL-STD-002
 - *Domain:* Requirements Management (REQM)
 - *Description:* User Story Specification & Bidirectional Traceability Rule
 - *Category:* Mandatory (All projects must maintain links between Epics, Stories, and Test Cases)
 - **Row 3:**
 - *Asset ID:* PAL-STD-003
 - *Domain:* Verification (VER)
 - *Description:* Definition of Done (DoD) & Peer Code Review Policy
 - *Category:* Mandatory (Code cannot merge without ≥ 1 peer approval and SonarQube scan)

- **Row 4:**
 - *Asset ID:* PAL-STD-004
 - *Domain:* Decision Analysis and Resolution (DAR)
 - *Description:* Architectural Trade-Off Scoring Matrix
 - *Category:* Tailorable (Evaluation criteria weights can be adjusted per business case)
- 7. Below the table, type /panel and select an **Info Panel**.
- 8. Set the panel title to CMMI Tailoring Policy:
 - Enter: Any deviation or tailoring of Mandatory PAL standards requires documented approval from the PMO and Lead Systems Architect, logged within the project's Confluence tailoring register prior to sprint execution.
- 9. Click **Publish** in the upper-right corner.

Activity 1.2: Automating CMMI Process Audits via Jira Automation Rules

- **Tool:** Jira Software
- **Objective:** Configure an automated Jira rule that enforces CMMI process compliance by validating that no issue moves to "In Progress" without a designated PAL Process Category tag.

Step-by-Step Instructions:

1. Open your project in Jira Software.
2. Click **Project Settings** (gear icon) in the bottom-left sidebar.
3. In the left-hand navigation menu, select **Automation**.
4. Click **Create rule** in the upper-right corner.
5. In the **Trigger** search bar, type Issue transitioned and select it.
6. Set **From status** to To Do (or Backlog) and **To status** to In Progress. Click **Save**.
7. Click **+ New condition**, then select **Issue fields condition**.
8. Set the condition parameters:
 - *Field:* Labels
 - *Condition:* does not contain any of
 - *Value:* Enter CMMI-Compliant
9. Click **Save**.
10. Click **+ New action**, then select **Transition issue**.
11. Set the target status to To Do (forcing the issue back out of active progress). Click **Save**.
12. Click **+ New action**, then select **Add comment**.
13. In the comment text field, enter:

AUTOMATED CMMI GOVERNANCE AUDIT: Transition blocked. This work item violates PAL-STD-002. You must ensure user story criteria meet organizational standards and apply the 'CMMI-Compliant' label before moving to In Progress.

14. Click **Save**.
15. In the rule summary panel on the right, name the rule: [Your Name] - CMMI Process Transition Enforcer.
16. Click **Turn on rule**.
17. Verify the rule by going to your board, creating a story without the label, dragging it to **In Progress**, and observing the automated reversion and comment.

2: Understanding Maturity Levels (1–5) and What They Mean for Project Governance

Activity 2.1: Designing the CMMI Maturity Progression Roadmap in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Construct a visual comparative maturity model in Miro that contrasts governance behaviors across Levels 1 through 5, highlighting operational transitions from ad hoc firefighting to statistical control.

Step-by-Step Instructions:

1. Open Miro and click **+ New board > Create a blank board**.
2. Title the board: [Your Name] - CMMI Maturity Progression (Levels 1-5).
3. Select the **Kanban / Table** tool from the left toolbar (or build 5 vertical columns using rectangles):
 - Column 1: Level 1: Initial
 - Column 2: Level 2: Managed
 - Column 3: Level 3: Defined
 - Column 4: Level 4: Quantitatively Managed
 - Column 5: Level 5: Optimizing
4. Add sticky notes in each column detailing operational reality and governance mechanisms:
 - **In Level 1 (Initial):**
 - Color: Red
 - Sticky 1: Behavior: Heroic effort, chaotic schedules, undocumented tribal knowledge.
 - Sticky 2: Governance: No formal stage gates; scope expands without budget reconciliation.
 - **In Level 2 (Managed):**
 - Color: Orange
 - Sticky 1: Behavior: Project-level discipline; work planned, tracked, and monitored per team.
 - Sticky 2: Governance: Basic requirements baselining, cost-to-schedule tracking, localized reviews.
 - **In Level 3 (Defined):**
 - Color: Yellow
 - Sticky 1: Behavior: Enterprise standard processes; tailored from the organization's PAL.
 - Sticky 2: Governance: Structured peer reviews, formal Decision Analysis (DAR), integrated risk logs.
 - **In Level 4 (Quantitatively Managed):**
 - Color: Light Green
 - Sticky 1: Behavior: Statistical Process Control (SPC); defect rates and velocity modeled quantitatively.
 - Sticky 2: Governance: Predictive process performance baselines; control charts for cycle time.
 - **In Level 5 (Optimizing):**
 - Color: Dark Green
 - Sticky 1: Behavior: Continuous, incremental, and innovative process improvement.

- Sticky 2: Governance: Automated root-cause defect prevention; dynamic pipeline re-engineering.
5. Draw a large progression arrow spanning the bottom of all columns from Level 1 to Level 5 with the label: Shift from Unpredictable Firefighting to Statistical Predictability.
 6. Export the board view as a PDF or high-resolution PNG for your project portfolio.

Activity 2.2: Tracking Level 4 Process Performance Baselines in Confluence

- **Tool:** Confluence
- **Objective:** Create a Level 4 Quantitatively Managed dashboard in Confluence that models statistical control limits for sprint velocity and defect containment.

Step-by-Step Instructions:

1. Open Confluence and navigate to your project space.
2. Click **Create** (+) and title the page: [Your Name] - CMMI Level 4 Quantitative Project Performance Baseline (QPM).
3. Add a brief summary paragraph: This register establishes historical Process Performance Models (PPM) to statistically forecast delivery predictability and flag process anomalies using 3-sigma thresholds.
4. Type /table and press **Enter** to insert a 6-column, 6-row table.
5. Populate the header row:
 - Sprint / Cadence
 - Planned Story Points
 - Delivered Points
 - Defect Leakage (UAT)
 - Upper Control Limit (UCL = +15%)
 - Lower Control Limit (LCL = -15%)
6. Insert historical baseline sample data across 5 iterations:
 - **Sprint 21:** Planned: 40 | Delivered: 39 | Defects: 2 | UCL: 46 | LCL: 34
 - **Sprint 22:** Planned: 42 | Delivered: 41 | Defects: 1 | UCL: 48.3 | LCL: 35.7
 - **Sprint 23:** Planned: 45 | Delivered: 30 | Defects: 8 | UCL: 51.7 | LCL: 38.2
 - **Sprint 24:** Planned: 35 | Delivered: 36 | Defects: 0 | UCL: 40.2 | LCL: 29.7
 - **Sprint 25:** Planned: 38 | Delivered: 37 | Defects: 2 | UCL: 43.7 | LCL: 32.3
7. Highlight **Sprint 23** in light red.
8. Below the table, type /panel and insert a **Warning Panel**.
9. In the panel, record the Level 4 Quantitative Governance Analysis:
 - OUT-OF-CONTROL EVENT (Sprint 23): Delivered points (30) fell below Lower Control Limit (38.2), correlated with an abnormal spike in Defect Leakage (8). Under CMMI Level 4 QPM, this variance triggers an immediate Causal Analysis and Resolution (CAR) review rather than casual team dismissal.
10. Click **Publish**.

3: Mapping CMMI to SDLC Phases and Stage Gates

Activity 3.1: Mapping CMMI Practice Areas to SDLC Gates in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Produce an architectural map connecting specific CMMI Practice Areas directly to SDLC phases and modern stage gates.

Step-by-Step Instructions:

1. Log in to Lucidchart and create a **Blank Document**.
2. Rename the document: [Your Name] - CMMI Process-to-SDLC Stage-Gate Blueprint.
3. Drag a **Process (Rectangle)** shape onto the left edge of the canvas:
 - Label: SDLC Phase 1: Inception & Feasibility
 - Sub-bullets inside shape: Scope Definition | Feasibility Analysis
4. Directly below it, add a secondary rectangle representing the associated CMMI Practice Areas:
 - Label: CMMI Practice Areas: Project Planning (PP) + Requirements Management (REQM)
 - Style: Dashed border, light blue fill.
5. To the right of Phase 1, insert a **Diamond (Decision)** shape:
 - Label: Stage Gate 1: Business Case & Scope Baseline
6. Repeat this pattern across the canvas for 3 additional phases:
 - **Phase 2: Architecture & Technical Spikes**
 - *CMMI Areas:* Technical Solution (TS) + Decision Analysis and Resolution (DAR)
 - *Gate 2:* Architecture & High-Risk Spike Baseline
 - **Phase 3: Sprints & Construction**
 - *CMMI Areas:* Product Integration (PI) + Verification (VER - Peer Reviews)
 - *Gate 3:* Definition of Done (DoD) Quality Gate
 - **Phase 4: UAT & Production Cutover**
 - *CMMI Areas:* Validation (VAL) + Configuration Management (CM)
 - *Gate 4:* Operational Readiness Review (ORR)
7. Connect each Phase box to its corresponding Gate diamond using directional solid lines.
8. Connect the CMMI dashed boxes to the main phase boxes using vertical connector lines.
9. Click **Share** > generate a viewable web link (or Export as PNG) for submission.

Activity 3.2: Implementing CMMI Verification & Stage-Gate Transition Rules in Jira

- **Tool:** Jira Software
- **Objective:** Enforce CMMI Verification (VER) stage-gate criteria directly within Jira by using workflow transition properties and resolution controls.

Step-by-Step Instructions:

1. In Jira, navigate to your project and click **Project Settings** (gear icon) > **Workflows**.
2. Click the **Pencil (Edit)** icon next to your primary development workflow.
3. Select the transition arrow between **In Review / Verification** and **Done**.
4. In the configuration sidebar on the right, select **Conditions**.
5. Click **Add condition**.
6. Select **Sub-Task Blocking Condition** and click **Add**.

7. Configure the condition:
 - Check the radio button: All sub-tasks must have one of the following statuses.
 - Select status: Done (or Closed).
8. Click **Add**.
9. Now click **Validators** on the same transition, then click **Add validator**.
10. Select **Field Required Validator** and select Resolution. Click **Add**.
11. Click **Publish Draft** in the workflow editor top banner (select No to saving a backup).
12. Return to your board and verify the governance rule:
 - Create a parent Story: Implement Single Sign-On (SSO) Module.
 - Add a sub-task: Conduct Peer Code Review (CMMI VER).
 - Leave the sub-task in To Do.
 - Attempt to drag the parent Story directly to Done.
 - Confirm that Jira blocks the transition, enforcing completion of verification artifacts.

4: How CMMI Strengthens Planning, Estimation, Risk Management, and Stakeholder Alignment

Activity 4.1: Building a Formal CMMI Decision Analysis and Resolution (DAR) Matrix in Confluence

- **Tool:** Confluence
- **Objective:** Formulate an objective, weighted Decision Analysis and Resolution (DAR) framework in Confluence to eliminate subjective bias and political friction from technical architectural decisions.

Step-by-Step Instructions:

1. Open your Confluence space, click **Create (+)**, and title the page: [Your Name] - CMMI DAR: Cloud API Gateway Selection.
2. Insert an introductory section using /heading 2: 1. Purpose and Context.
3. Add text: Documenting the objective evaluation for selecting an enterprise-grade API gateway under CMMI DAR guidelines, aligning security, latency, and operational cost against project baselines.
4. Type /table and insert a 6-column, 5-row evaluation matrix.
5. Label the column headers:
 - Evaluation Criteria
 - Weight (1–5)
 - Candidate A: Kong Gateway (Score 1–5)
 - Candidate A: Weighted Score
 - Candidate B: AWS API Gateway (Score 1–5)
 - Candidate B: Weighted Score
6. Fill in the evaluation rows:
 - **Row 1:**
 - Criteria: Latency & Throughput SLA (<20ms) | Weight: 5
 - Candidate A: 5 | Weighted: 25
 - Candidate B: 4 | Weighted: 20
 - **Row 2:**
 - Criteria: On-Premise / Hybrid Portability | Weight: 4

- Candidate A: 5 | Weighted: 20
- Candidate B: 2 | Weighted: 8
- **Row 3:**
 - Criteria: Ease of Integration & Maintenance | Weight: 3
 - Candidate A: 3 | Weighted: 9
 - Candidate B: 5 | Weighted: 15
- **Row 4:**
 - Criteria: Licensing & Operational Cost | Weight: 4
 - Candidate A: 4 | Weighted: 16
 - Candidate B: 3 | Weighted: 12
- 7. Add a bottom total row:
 - Criteria: **TOTALS** | Weight: -- | Cand A: -- | **Cand A Total: 70** | Cand B: -- | **Cand B Total: 55**
- 8. Below the table, use /panel to add a **Success Panel**:
 - Heading: Formal DAR Selection & Stakeholder Rationale
 - Text: Kong Gateway selected with an objective score of 70 vs. 55. High hybrid portability and latency compliance outweigh AWS API Gateway's native managed convenience, satisfying regulatory hosting mandates.
- 9. Click **Publish**.

Activity 4.2: Implementing a Quantitative Risk Breakdown Structure (RSKM) in Jira

- **Tool:** Jira Software
- **Objective:** Implement a CMMI-compliant Risk Management (RSKM) framework in Jira that quantifies risk exposure and establishes automated risk mitigation alerts.

Step-by-Step Instructions:

1. In Jira, navigate to your project and select **Project Settings > Issue Types**.
2. Click **Add issue type**, enter Project Risk, select a hazard/shield icon, and click **Create**.
3. Go to **Project Settings > Fields**.
4. Check that the issue type displays the fields: Summary, Description, Priority, and Due date.
5. Return to your project board and click **+ Create** in the main navigation menu.
6. Create Risk Ticket 1:
 - *Issue Type:* Project Risk
 - *Summary:* RSKM-01: Third-Party Authentication API Rate Limiting
 - *Priority:* High
 - *Description:*
 - *Probability (1–5):* 4
 - *Impact (1–5):* 4
 - *Total Risk Exposure Score ($P \times I$):* 16 (High Exposure)
 - *Mitigation Trigger:* If third-party latency exceeds 500ms during staging load tests.
 - *Planned Mitigation:* Implement Redis distributed token caching layer.
7. Click **Create**.
8. Open the newly created risk ticket RSKM-01.
9. In the **Linked Issues** field, select relates to (or causes risk to), then select an existing functional user story from your backlog.
10. Click the **Actions (...)** menu on the top-right of the ticket and select **Add flag**.

11. Verify that the linked functional story shows the active risk dependency, establishing explicit CMMI bidirectional risk traceability.

5: Integrating CMMI with Agile, Scrum, and Hybrid Delivery Models

Activity 5.1: Mapping Scrum Ceremonies to CMMI Practice Areas in Confluence

- **Tool:** Confluence
- **Objective:** Create a cross-reference matrix that proves how standard Scrum sprint ceremonies directly fulfill formal CMMI Level 2 and Level 3 process area requirements without overhead.

Step-by-Step Instructions:

1. Open Confluence, navigate to your space, click **Create (+)**, and title the page: [Your Name] - Agile-to-CMMI Process Cross-Reference Standard.
2. Type /table and insert a 4-column, 5-row table.
3. Label the headers:
 - Scrum Ceremony / Artifact
 - Agile Purpose
 - Equivalent CMMI Practice Area
 - Mandatory Audit Evidence Required
4. Enter the ceremony mappings into the rows:
 - **Row 1:**
 - *Scrum:* Sprint Planning & Commitment
 - *Agile Purpose:* Establish Sprint Goal and select user stories based on velocity.
 - *CMMI:* Project Planning (PP) & Requirements Management (REQM)
 - *Audit Evidence:* Sprint backlog snapshot in Jira; capacity allocation record.
 - **Row 2:**
 - *Scrum:* Daily Stand-up
 - *Agile Purpose:* Synchronize progress, inspect flow, and surface impediments.
 - *CMMI:* Project Monitoring and Control (PMC)
 - *Audit Evidence:* Jira Active Sprint board with flagged blocked issues and updated sub-tasks.
 - **Row 3:**
 - *Scrum:* Sprint Review & Demonstration
 - *Agile Purpose:* Demonstrate working software increment to business stakeholders for feedback.
 - *CMMI:* Validation (VAL) & Product Integration (PI)
 - *Audit Evidence:* Recorded demo session; accepted acceptance criteria check-offs in Jira.
 - **Row 4:**
 - *Scrum:* Sprint Retrospective
 - *Agile Purpose:* Inspect team process and agree on immediate improvements.
 - *CMMI:* Causal Analysis and Resolution (CAR)
 - *Audit Evidence:* Confluence Retrospective log with assigned action items linked to the next sprint backlog.

5. Below the table, insert an **Info Panel** (/panel):
 - Heading: Auditor Compliance Note
 - Text: Under CMMI Level 3, external auditors accept Jira sprint histories, Git pull request logs, and Confluence retrospective notes as sufficient proof of process institutionalization, rendering separate administrative paperwork unnecessary.
6. Click **Publish**.

Activity 5.2: Configuring an Integrated Agile-CMMI Sprint Board in Jira

- **Tool:** Jira Software
- **Objective:** Configure a Jira Scrum board layout with custom swimlanes and quick filters to track both Agile user stories and CMMI compliance tasks simultaneously.

Step-by-Step Instructions:

1. Open Jira and go to your Scrum board.
2. In the top-right corner, click the **three dots (...)** and select **Board settings**.
3. Select **Swimlanes** from the left navigation panel.
4. Set the **Base Swimlanes on** dropdown to Queries.
5. Add the following 3 swimlanes:
 - **Swimlane 1:**
 - *Name:* CMMI Blockers & Escalated Risks
 - *JQL Query:* issueType = "Project Risk" OR flag = "Impediment"
 - Click **Add**.
 - **Swimlane 2:**
 - *Name:* Standard Product Increments (User Stories)
 - *JQL Query:* issueType in (Story, Epic, Task) AND (flag is EMPTY OR flag != "Impediment")
 - Click **Add**.
 - **Swimlane 3:**
 - *Name:* Quality & Process Governance (CMMI PPQA/VER)
 - *JQL Query:* issueType in (Bug, "Stage Gate Review")
 - Click **Add**.
6. Select **Quick Filters** from the board settings left menu.
7. Create two dedicated filters:
 - *Filter 1 Name:* CMMI Traceable Items
 - *JQL:* issueType = Story AND description ~ "Given" AND description ~ "Then"
 - *Filter 2 Name:* Missing Verification
 - *JQL:* issueType = Story AND subtasks is EMPTY
8. Click **Add** for both filters.
9. Click **Back to board** in the top left.
10. Toggle the **Missing Verification** quick filter to instantly highlight any sprint user story that lacks verified sub-tasks or code review steps.

Agile & Scrum for Project Managers: Individual Hands-on Lab Guide

1: Navigating Sprints, Daily Stand-Ups, and Retrospectives

Activity 1.1: Facilitating an Asynchronous Daily Stand-Up System in MS Teams

- **Tool:** MS Teams
- **Objective:** Configure an asynchronous daily stand-up channel workflow that captures operational blockers, enforces daily flow governance, and preserves audit trails for CMMI Project Monitoring and Control (PMC).

Step-by-Step Instructions:

1. Open MS Teams and navigate to your team or create a sandbox team via **Join or create a team > Create team > From scratch > Private**. Name it [Your Name] - Agile Delivery Team.
2. Click the three dots (...) next to your team name and select **Add channel**.
3. In the channel name field, enter: daily-standup-async.
4. Set the privacy to **Standard** and click **Add**.
5. Click the three dots (...) next to the newly created channel and select **Manage channel**.
6. Under **Channel moderation**, toggle moderation to **On** (or ensure permissions restrict post creation to top-level announcements to avoid thread clutter).
7. Return to the channel posts area and click **New conversation** (or **Post**).
8. Click the **Format** icon (pen with a letter A) below the message box to expand the rich editor, and switch the post type dropdown from *Conversation* to **Announcement**.
9. In the large headline banner, enter: DAILY STAND-UP PROTOCOL & AUDIT LOG.
10. In the subheader, type: Mandatory daily sync window: 09:00 - 09:30 AM local time.
11. In the body of the post, copy and paste this standard template:

****Operational Check-In Template (Reply to this thread directly):****

1. ****Committed Value Yesterday:**** [Jira Ticket ID - Summary]
2. ****Targeted Value Today:**** [Jira Ticket ID - Summary]
3. ****Blockers / Process Friction:**** [None OR Detail with @mentions]
4. ****CMMI Verification Flag:**** [Have all in-progress sub-tasks been updated? (Yes/No)]

12. Click the paper airplane icon (**Post**).
13. Click **Reply** directly under your announcement post and submit a simulated developer entry:
 - *Yesterday:* PAY-101 - Integrated OAuth token refresh endpoint
 - *Today:* PAY-104 - Building multi-currency transaction validator
 - *Blocker:* Blocked by third-party sandbox certificate expiration. @ModernPM please escalate.
 - *CMMI Flag:* Yes, code review sub-task logged.
14. Click the three dots (...) on your reply, select **Pin**, and confirm the pin to ensure the escalation remains visible at the top of the channel.

Activity 1.2: Running a CMMI CAR Retrospective Board in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Design an interactive sprint retrospective board that links process retrospection directly to CMMI Causal Analysis and Resolution (CAR) action tracking.

Step-by-Step Instructions:

1. Open Miro and click + **New board** > **Create a blank board**.
2. Rename the board in the top left: [Your Name] - Sprint Retrospective & CAR Engine.
3. Use the **Shape** tool (R) to draw 4 large vertical rectangles side by side.
4. Set the header labels for each column:
 - Column 1: What Went Well (Process Strengths)
 - Column 2: Where Did Delivery Friction Occur? (Impediments)
 - Column 3: Root-Cause Deep Dive (5-Whys Analysis)
 - Column 4: CMMI CAR Action Items (Next Sprint Backlog)
5. Style Column 1 light green, Column 2 light red, Column 3 light yellow, and Column 4 light blue.
6. In Column 2, place a red sticky note (N):
 - Text: Escaped regression bug breached staging environment during Sprint 4 cutover.
7. In Column 3, place 3 connected yellow sticky notes arranged vertically to model a 5-Whys chain:
 - *Why 1:* Staging build broke because database migration script lacked rollback instructions.
 - *Why 2:* Developer executed manual schema changes without automated CI pipeline execution.
 - *Why 3:* Definition of Done did not mandate automated schema migration checks before PR approval.
8. Draw a directional arrow connecting the red sticky note in Column 2 to the top yellow sticky note in Column 3.
9. In Column 4, add a blue sticky note representing the corrective policy:
 - Text: CAR-ACT-04: Update organizational Definition of Done (DoD) in Confluence to require Liquibase automated schema test execution on all pull requests.
10. Draw a directional arrow from the final yellow "Why" sticky in Column 3 to the blue CAR action note in Column 4, establishing a clear cause-and-effect remediation path.

2: Translating Business Requirements into Epics and User Stories

Activity 2.1: Mapping Strategic Business Requirements to User Stories in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Deconstruct high-level, ambiguous business requirements into a functional Story Map that maintains hierarchical traceability across Epics, Personas, and INVEST-compliant User Stories.

Step-by-Step Instructions:

1. Open Miro, click + **New board**, and title it: [Your Name] - User Story Map & Traceability Architecture.
2. Across the top of the board, create a horizontal banner using a dark blue rectangle labeled:
 - Strategic Business Goal: Automated Cross-Border Corporate Settlements Engine

3. Below the banner, create a horizontal row of 3 gray rounded rectangles to represent the **User Journey Backbone**:
 - Backbone Step 1: Account Authentication & KYC Validation
 - Backbone Step 2: Transaction Initiation & Forex Rate Locking
 - Backbone Step 3: Settlement Processing & Regulatory Reporting
4. Directly beneath each backbone step, place a purple sticky note representing the **Epic Level**:
 - Below Step 1: Epic 1: Multi-Factor Enterprise Authentication
 - Below Step 2: Epic 2: Real-Time Forex Quoting Engine
 - Below Step 3: Epic 3: Automated AML & Compliance Filing
5. Beneath **Epic 2 (Real-Time Forex Quoting Engine)**, add 3 vertical sticky notes (light green) detailing vertical **User Stories**:
 - *Story 1*: As a Corporate Treasurer, I want real-time FX rate streaming so that I can execute trades at market parity.
 - *Story 2*: As a Corporate Treasurer, I want a 60-second rate lock window so that volatility does not break transaction terms.
 - *Story 3*: As a Risk Auditor, I want expired rate locks automatically rejected so that slippage risk is minimized.
6. Click on **Story 2**, then select the **Link** icon in the context toolbar. Paste a placeholder link to your Jira project or Confluence requirement page to represent CMMI Level 3 bidirectional requirement traceability.
7. Add a small text label beneath Story 2 showing INVEST validation: [I]ndependent, [N]egotiable, [V]aluable, [E]stimable, [S]mall, [T]estable.

Activity 2.2: Building Epics, User Stories, and Gherkin Acceptance Criteria in Jira

- **Tool**: Jira Software
- **Objective**: Create and structure an Epic-to-Story hierarchy in Jira, embedding formal Given-When-Then acceptance criteria that satisfy CMMI Requirements Management (REQM) audit standards.

Step-by-Step Instructions:

1. Navigate to your project in Jira Software.
2. Click **+ Create** in the top navigation bar.
3. Configure the parent Epic:
 - *Issue Type*: Epic
 - *Epic Name*: Corporate Forex Engine
 - *Summary*: Implement Real-Time FX Rate Quoting and Lock System
 - *Description*: Business objective: Enable institutional clients to secure live spot rates with automated margin calculations.
4. Click **Create**.
5. Click **+ Create** again to create the child User Story:
 - *Issue Type*: Story
 - *Summary*: As a Treasurer, I want a 60-second FX rate lock to execute volatility-free trades
 - *Epic Link / Parent*: Search and select Corporate Forex Engine.
 - *Story Points*: 5

6. In the **Description** field of the Story, copy and paste this formatted Gherkin specification:

Business Value & Traceability:

- Business Driver: CMMI-REQ-FX-002 (Treasury Automation)

Acceptance Criteria (Given-When-Then):

Scenario 1: Successful Rate Lock Initiation

Given the user is authenticated as an authorized Corporate Treasurer

When the user requests a spot rate quote for EUR/USD

Then the system displays the mid-market rate with a visible 60-second countdown timer.

Scenario 2: Expired Rate Lock Execution Attempt

Given the 60-second countdown timer has reached 0

When the user attempts to confirm the trade execution

Then the system rejects the transaction

And displays the error message: "Quote expired. Please refresh for a live rate."

7. Click **Create**.
8. Navigate to your **Backlog** view from the left menu.
9. Verify that the newly created User Story appears directly nested under the parent Epic in the backlog list.

3: Velocity Tracking, Story Pointing, and Capacity Planning

Activity 3.1: Building a Dynamic Sprint Capacity & Commitment Calculator in Confluence

- **Tool:** Confluence
- **Objective:** Author a data-driven capacity planning model in Confluence that translates individual engineer availability into net productive hours and recommended story point commitments.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - Sprint 14 Capacity Plan & Commitment Baseline.
3. Type /table and insert a 6-column, 6-row table.
4. Set the header row titles:
 - Team Member Name
 - Role
 - Working Days in Sprint
 - Time Off / Holidays (Days)
 - Focus Factor % (Operational Buffer)
 - Net Productive Engineering Days
5. Populate the table with sample engineering capacity data:
 - **Row 1:** Dev Lead | Lead Backend Engineer | 10 | 0 | 60% | 6.0
 - **Row 2:** Frontend Dev 1 | UI Engineer | 10 | 1 | 75% | 6.75
 - **Row 3:** Backend Dev 2 | Data Engineer | 10 | 2 | 75% | 6.0
 - **Row 4:** QA Engineer 1 | SDET / Automation | 10 | 0 | 75% | 7.5
 - **Row 5: TOTALS** | -- | 40 | 3 | -- | **26.25 Net Days**

6. Below the table, use /heading 2 to add: Capacity vs. Historical Velocity Reconciler.
7. Type /table and insert a 2-column, 4-row summary table:
 - Row 1: Historical 3-Sprint Rolling Average Velocity | 34 Story Points
 - Row 2: Historical Team Daily Point Conversion Rate | 1.3 Points / Productive Day
 - Row 3: Theoretical Capacity Ceiling (26.25 Net Days x 1.3) | 34.1 Points
 - Row 4: Recommended Sprint 14 Commitment Target | **32 Story Points (Safe Baseline)**
8. Below the summary table, insert an **Expand** block (/expand) titled: CMMI Project Planning (PP) Justification.
9. Inside the expand block, type: The team rejected stakeholder demands to pull 42 points into Sprint 14 because net available days dropped from 30 to 26.25 due to scheduled leaves. Committing beyond 32 points violates CMMI Level 2/3 planning discipline by inducing unmitigated defect leakage.
10. Click **Publish**.

Activity 3.2: Configuring Story Point Estimation and Velocity Tracking in Jira

- **Tool:** Jira Software
- **Objective:** Configure a Jira Scrum board's estimation statistics, populate a sprint backlog with pointed user stories, and evaluate velocity forecasting reports.

Step-by-Step Instructions:

1. Navigate to your Jira Scrum project.
2. In the top-right corner of the board view, click the three dots (...) and select **Board settings**.
3. Select **Estimation** from the left-hand navigation list.
4. Set the **Estimation statistic** dropdown to Story Points.
5. Under **Time Tracking**, select None (ensuring the board stays focused on relative complexity sizing rather than hourly micromanagement).
6. Click **Back to board** in the top left corner, then navigate to the **Backlog** view.
7. Click **Create sprint** at the top of the backlog list. A container named Sprint 1 (or next sequential sprint) will appear.
8. Click **+ Create issue** within the sprint container to add 4 distinct user stories:
 - Story A: Implement User Registration API -> Assign **Story Points: 3**
 - Story B: Build Multi-Currency Ledger Database Schema -> Assign **Story Points: 8**
 - Story C: Integrate Stripe Webhook Listeners -> Assign **Story Points: 5**
 - Story D: Automate KYC Verification Document Upload -> Assign **Story Points: 13**
9. Review the top right of your sprint header: verify that the gray circle indicates a total committed scope of **29 Story Points**.
10. Click **Start sprint** on the sprint header.
11. In the modal, verify the Sprint Name, set the **Duration** to 2 weeks, enter a **Sprint Goal** (Deliver core registration and ledger integration baseline), and click **Start**.
12. In the left-hand navigation menu, click **Reports**.
13. Select **Velocity Chart** from the available agile reports list.
14. Inspect the velocity visualization: verify how Jira plots **Commitment** vs. **Completed** story points across closed sprints, establishing the quantitative baseline for future sprint planning.

Requirement Gathering and Analysis: Individual Hands-on Lab Guide

1: Collaborating with Stakeholders to Identify Edge Cases Early

Activity 1.1: Running an Example Mapping & Edge-Case Discovery Workshop in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Lead an Example Mapping session to uncover operational boundaries, concurrency issues, and negative scenarios before user stories enter development.

Step-by-Step Instructions:

1. Open Miro and click **+ New board > Create a blank board**.
2. Rename the board in the upper-left corner: [Your Name] - Example Mapping & Edge-Case Matrix.
3. Add a title banner across the top using the **Text** tool (T): Example Mapping: Digital Wallet Funds Transfer Feature.
4. Place a large yellow sticky note (N) at the top center to represent the **User Story**:
 - Text: Story: As an account holder, I want to transfer funds instantly to another user so that I can settle peer debts.
5. Below the story card, create 3 vertical columns using the **Shape** tool (R) or Miro cards:
 - Column 1: Business Rules (Blue Sticky Notes)
 - Column 2: Concrete Examples & Edge Cases (Green Sticky Notes)
 - Column 3: Unresolved Questions & System Risks (Red Sticky Notes)
6. Populate **Column 1** with 2 blue sticky notes representing business rules:
 - *Rule 1 (Blue)*: Sender must maintain a cleared ledger balance equal to or greater than transfer amount + transaction fee.
 - *Rule 2 (Blue)*: Daily cumulative peer-to-peer transfers cannot exceed \$2,500.
7. Populate **Column 2** with 3 green sticky notes illustrating concrete examples and boundary edge cases:
 - *Example 1 - Happy Path (Green)*: Sender has \$500 balance, sends \$100 with \$0 fee. Transfer succeeds; sender balance becomes \$400.
 - *Example 2 - Boundary Edge Case (Green)*: Sender has \$2,500 daily limit, has spent \$2,450 today, and attempts a \$50 transfer. Transfer succeeds exactly at the \$2,500 threshold.
 - *Example 3 - Negative Concurrency Edge Case (Green)*: Sender has \$100 balance and fires two simultaneous \$100 transfers from web and mobile sessions within 200ms. Database row locking allows the first request and aborts the second with "Insufficient Funds".
8. Populate **Column 3** with 2 red sticky notes capturing unaddressed operational ambiguities:
 - *Question 1 (Red)*: How does the system behave if the recipient account is closed or frozen mid-flight?
 - *Question 2 (Red)*: What is the automated timeout retry window if the core settlement ledger returns HTTP 504?
9. Use the **Line** tool (L) to connect each green example and red question card directly to its parent blue business rule card.
10. Export the completed board view as a PNG or copy the shareable link for submission.

Activity 1.2: Embedding Edge-Case Scenarios into Jira User Stories via BDD (Gherkin)

- **Tool:** Jira Software
- **Objective:** Formalize edge cases and system boundaries directly inside Jira user stories using Given-When-Then acceptance criteria, fulfilling CMMI Requirements Development (RD) and Verification (VER) standards.

Step-by-Step Instructions:

1. Open Jira Software and navigate to your project backlog.
2. Click **+ Create** in the top navigation bar.
3. Configure the story issue:
 - *Issue Type:* Story
 - *Summary:* Implement Peer-to-Peer Funds Transfer Execution Engine
 - *Component:* Payments API (or leave default)
 - *Story Points:* 8
4. In the **Description** field, copy and paste the following structured specification:

Requirement Traceability ID:

- Business Baseline: CMMI-RD-PAY-04

Primary Scenario (Happy Path):

Scenario: Standard valid transfer between active accounts

Given Sender account "ACC-01" has a cleared balance of \$500.00

And Recipient account "ACC-02" is active and KYC-cleared

When Sender submits an instant transfer of \$150.00

Then Debit \$150.00 from "ACC-01"

And Credit \$150.00 to "ACC-02"

And Issue confirmation transaction receipt "TXN-SUCCESS".

Shift-Left Edge-Case Scenarios:

Scenario: Concurrency lock / Race condition prevention

Given Sender account "ACC-01" has an active cleared balance of \$100.00

When Two simultaneous transfer requests of \$100.00 each are received within 150ms

Then The ledger executes an atomic row lock on "ACC-01"

And Exactly one transaction completes successfully

And The second transaction is rejected with error code "ERR-4009: Insufficient Cleared Funds"

And Total debited amount does not exceed \$100.00.

Scenario: Upstream Core Banking Settlement Timeout (HTTP 504)

Given Sender account "ACC-01" has sufficient funds

When The core payment switch does not respond within 3,000ms

Then The transaction state transitions to "PENDING_RECONCILIATION"

And No funds are deducted from the customer's available ledger balance

And An automated status inquiry job is scheduled for execution at 60-second intervals.

5. Click **Create**.
6. Open the newly created ticket on your backlog/board.

7. Click the **Actions (...)** menu in the top-right corner of the issue window and select **Add flag**.
8. Add a comment: Refinement Note: Concurrency and timeout edge cases reviewed with Lead Architect and QA Lead. Story meets Definition of Ready (DoR).
9. Click **Save** on the comment.

2: Defining the Minimum Viable Product (MVP) and Release Phases

Activity 2.1: Slicing an MVP and Multi-Release Phasing Map in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Visually architect an MVP and subsequent delivery releases using horizontal slicing techniques, aligning technical deliverables with business value milestones.

Step-by-Step Instructions:

1. Log in to Lucidchart and click **+ New > Lucidchart > Blank Document**.
2. Title the document: [Your Name] - Product MVP Slicing & Release Phasing Architecture.
3. Drag a **Horizontal Swimlane** container onto the canvas and configure it with **3 lanes**:
 - Label Swimlane 1: Phase 1: Minimum Viable Product (MVP - Release 1.0)
 - Label Swimlane 2: Phase 2: Enhanced Automation & Scale (Release 2.0)
 - Label Swimlane 3: Phase 3: Enterprise & Optimization (Release 3.0)
4. Set the header background of Swimlane 1 to light blue, Swimlane 2 to light yellow, and Swimlane 3 to light gray.
5. In **Swimlane 1 (MVP)**, place 4 process rectangles across the lane representing core end-to-end user journeys:
 - Block A: Basic Email & Password Authentication
 - Block B: Direct Account-to-Account Transfer (USD Only)
 - Block C: Simple Transaction History View (Last 30 Days)
 - Block D: Manual CSV Admin Reconciliation Tool
6. In **Swimlane 2 (Release 2.0)**, place 3 process rectangles aligned beneath the MVP features:
 - Block A: Biometric Login (FaceID / Fingerprint) & MFA
 - Block B: Multi-Currency Transfers & Real-Time FX Rate Locks
 - Block C: Exportable PDF Statements & Push Notifications
7. In **Swimlane 3 (Release 3.0)**, place 3 process rectangles:
 - Block A: Single Sign-On (SSO / SAML) for Corporate Accounts
 - Block B: Automated Split-Payment API & Recurring Subscriptions
 - Block C: AI-Powered Anomaly & Fraud Scoring Engine
8. Draw vertical dashed divider lines cutting downward through all three lanes to represent functional domains:
 - Divider 1: Identity & Security
 - Divider 2: Core Payments Engine
 - Divider 3: Reporting & Administration
9. Add a callout bubble attached to Swimlane 1 with the note: CMMI Baseline: Phase 1 (MVP) contains only "Must-Have" stories required to prove product viability while maintaining zero unmitigated security risks.
10. Click **File > Export > PNG** (or copy the shareable document link) for submission.

Activity 2.2: Partitioning Backlogs into Phased Fix Versions in Jira

- **Tool:** Jira Software
- **Objective:** Structure an enterprise backlog into formal release versions, apply MoSCoW prioritization tags, and audit release candidate scope boundaries.

Step-by-Step Instructions:

1. In Jira, navigate to your project and select **Releases** from the left-hand navigation sidebar.
2. Click **Create version** in the upper-right corner.
3. Configure the MVP release candidate:
 - *Name:* Release 1.0 - MVP
 - *Start date:* Select today's date.
 - *Release date:* Select a date 4 weeks from today.
 - *Description:* Minimum Viable Product: Core account transfers and compliance logging.
4. Click **Save**.
5. Click **Create version** again to configure the second phase:
 - *Name:* Release 2.0 - Scale & Automation
 - *Start date:* Select the day after Release 1.0 ends.
 - *Release date:* Select a date 8 weeks from today.
 - *Description:* Enhanced notifications, multi-currency support, and biometric authentication.
6. Click **Save**.
7. Navigate to the **Backlog** view from the left menu.
8. Click **+ Create issue** at the bottom of the backlog to add 4 distinct user stories:
 - Story 1: Core USD Transaction Processing Service
 - Story 2: Automated Daily Ledger Reconciliation Script
 - Story 3: Multi-Currency Real-Time Exchange Rate Calculator
 - Story 4: Automated Fraud Suspicion Machine Learning Classifier
9. Apply the **MoSCoW** label and assign the **Fix Version** to each story:
 - Click into **Story 1**: Add label Must-Have, set **Fix version** to Release 1.0 - MVP.
 - Click into **Story 2**: Add label Must-Have, set **Fix version** to Release 1.0 - MVP.
 - Click into **Story 3**: Add label Should-Have, set **Fix version** to Release 2.0 - Scale & Automation.
 - Click into **Story 4**: Add label Could-Have, set **Fix version** to Release 2.0 - Scale & Automation.
10. In the left panel of the backlog, click the **Versions** panel toggle (or open the **Versions** filter dropdown).
11. Click on Release 1.0 - MVP to filter the backlog: verify that only Stories 1 and 2 appear, demonstrating an auditable, bounded MVP scope baseline.

Risk Management & Technical Debt: Individual Hands-on Lab Guide

1: Identifying and Prioritizing Project and Technical Risks

Activity 1.1: Building an Interactive 5x5 Quantitative Risk Matrix in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Construct a dynamic 5x5 Probability vs. Impact Risk Matrix, plot project and architectural risks, calculate quantitative exposure scores ($P \times I$), and define automated mitigation triggers to fulfill CMMI Risk Management (RSKM) standards.

Step-by-Step Instructions:

1. Open Miro and click **+ New board > Create a blank board**.
2. Title the board in the upper-left corner: [Your Name] - CMMI Quantitative Risk Matrix & Triggers.
3. Select the **Table** tool from the left toolbar, click on the canvas, and configure a **5-row by 5-column grid**.
4. Set the axis labels:
 - **Vertical Axis (Left - Probability):** Label rows from bottom to top as 1 - Rare, 2 - Unlikely, 3 - Moderate, 4 - Likely, 5 - Almost Certain.
 - **Horizontal Axis (Bottom - Impact):** Label columns from left to right as 1 - Negligible, 2 - Minor, 3 - Moderate, 4 - Major, 5 - Critical.
5. Color-code the table cells to create risk severity heat zones:
 - Low Risk cells ($P \times I = 1$ to 4): Fill with light green.
 - Medium Risk cells ($P \times I = 5$ to 9): Fill with light yellow.
 - High Risk cells ($P \times I = 10$ to 14): Fill with light orange.
 - Critical Risk cells ($P \times I = 15$ to 25): Fill with light red.
6. Create 3 distinct risk sticky notes (N) and position them on the matrix:
 - **Sticky 1 (Critical Red Zone - Row 4, Col 4 | Score: 16):**
 - Text: RSKM-01: Core Banking Settlement API rate limit throttles during peak load (>500 req/sec).
 - **Sticky 2 (High Orange Zone - Row 3, Col 4 | Score: 12):**
 - Text: RSKM-02: Cloud infrastructure provider data sovereignty compliance audit delays staging sign-off.
 - **Sticky 3 (Medium Yellow Zone - Row 2, Col 3 | Score: 6):**
 - Text: RSKM-03: Senior Front-End Engineer turnover creates single-point dependency on checkout UI.
7. To the right of the matrix, create a rectangular container titled CMMI RSKM Operational Action & Mitigation Log.
8. Add a detailed mitigation card for **RSKM-01** inside this container:
 - *Risk Exposure:* 16 (High Exposure)
 - *Mitigation Strategy:* Architectural Spike in Sprint 5 to implement a Redis distributed queue buffer.
 - *Quantitative Trigger:* If staging load-test error rate exceeds 1.5% under 400 req/sec.
 - *Risk Owner:* Lead Systems Architect
9. Use the **Line** tool (L) to draw a red connector line from Sticky 1 on the matrix directly to its mitigation card in the log.

10. Click **Export this board** > **Save as image (PNG)** or generate a shareable view link for submission.

Activity 1.2: Configuring a Risk-Adjusted Backlog and Exposure Automation in Jira

- **Tool:** Jira Software
- **Objective:** Configure a dedicated Risk issue architecture in Jira with custom quantitative scoring fields, calculate total risk exposure via automated formulas, and link risks directly to sprint user stories.

Step-by-Step Instructions:

1. Navigate to your Jira Software project.
2. Click **Project settings** (gear icon in the bottom-left sidebar) > **Issue types**.
3. Confirm an issue type named Project Risk exists (if not, click **Add issue type**, enter Project Risk, select a hazard/shield icon, and click **Create**).
4. Click **Project settings** > **Automation** from the left navigation panel.
5. Click **Create rule** in the top-right corner.
6. Set the **Trigger**:
 - Type Field value changed in the search bar and select it.
 - In the **Fields to monitor** dropdown, select Priority (or custom number fields Risk Probability and Risk Impact if configured). Click **Save**.
7. Click **+ New condition**, then select **Issue fields condition**:
 - *Field:* Issue Type
 - *Condition:* equals
 - *Value:* Project Risk
 - Click **Save**.
8. Click **+ New action**, then select **Add comment**:
 - Enter the automated governance audit text:
 - CMMI RSKM AUTOMATION: Risk severity recalibrated.
 - Review mitigation contingency plans and verify that linked functional user stories have corresponding architectural spike buffers scheduled.
 - Click **Save**.
9. Click **+ New action**, select **Add flag**, and click **Save**.
10. Name the rule: [Your Name] - CMMI Risk Exposure Recalibration.
11. Click **Turn on rule**.
12. Test the configuration:
 - Click **+ Create** in the top navigation bar.
 - *Issue Type:* Project Risk
 - *Summary:* API Gateway Memory Leak Under High Concurrency
 - *Priority:* High
 - *Description:* Probability: 4 | Impact: 4 | Exposure: 16. Requires memory profiling spike.
 - Click **Create**.
13. Open the newly created Project Risk ticket.
14. Under **Linked Issues**, select causes risk to (or relates to), search for and select an active sprint user story (e.g., your payment/checkout story), and click **Link**.
15. Verify that the parent story now visibly displays an active linked technical risk, establishing the bidirectional risk traceability required by CMMI Level 3.

2: Balancing Feature Delivery with Technical Debt Repayment

Activity 2.1: Designing a 20% Technical Debt Allocation & Capacity Model in Confluence

- **Tool:** Confluence
- **Objective:** Author a data-driven Sprint Allocation Policy that establishes a dedicated 20% capacity budget for technical debt refactoring and proves mathematically how unmanaged debt degrades feature velocity over time.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create** (+).
2. Title the page: [Your Name] - Technical Debt Governance & Capacity Allocation Policy.
3. Insert an introductory section using /heading 2: 1. Executive Policy: The 80/20 Capacity Rule.
4. Add text: To prevent architectural decay, maintain CMMI Level 3 Verification standards, and eliminate defect escalation, every development sprint enforces a mandatory split between new feature delivery, architectural debt refactoring, and defect resolution.
5. Type /table and press **Enter** to insert a 4-column, 5-row allocation model table.
6. Populate the headers:
 - Work Category
 - Target Capacity Allocation %
 - Sprint Story Point Budget (Based on 40-pt Velocity)
 - Governance Criteria & CMMI Alignment
7. Fill in the table rows:
 - **Row 1:**
 - *Category:* New Feature Delivery (Business Epics)
 - *Allocation:* 70%
 - *Point Budget:* 28 Points
 - *Governance:* User-facing stories meeting Definition of Ready (DoR); aligns with Requirements Management (REQM).
 - **Row 2:**
 - *Category:* Technical Debt Refactoring & Upgrades
 - *Allocation:* 20%
 - *Point Budget:* 8 Points
 - *Governance:* Framework upgrades, database indexing, dead code removal, architecture spikes; aligns with Technical Solution (TS).
 - **Row 3:**
 - *Category:* Unplanned Defect / Escaped Bug Buffer
 - *Allocation:* 10%
 - *Point Budget:* 4 Points
 - *Governance:* In-sprint bug remediation and Sev 2 defect triage; aligns with Verification (VER).
 - **Row 4:**
 - *Category:* **TOTAL COMMITMENT**
 - *Allocation:* **100%**
 - *Point Budget:* **40 Points**
 - *Governance:* Total committed work must not exceed historical rolling velocity baseline.

8. Below the table, use /heading 2 to add: 2. Cost of Unmanaged Technical Debt (The Drag Coefficient).
9. Type /table to insert a 3-column, 4-row decay projection table showing stakeholders why technical debt cannot be deferred:
 - Operating Quarter | Debt Repayment Strategy | Net Velocity (Output to Market)
 - Q1 | Healthy 20% Allocation Enforced | 38 - 42 Story Points / Sprint
 - Q2 | 0% Debt Allocation (100% Features Pushed) | 35 Story Points / Sprint (Code complexity rises)
 - Q3 | Unmanaged Debt Death Spiral | 18 Story Points / Sprint (60% time lost to firefighting bugs)
10. Below the decay table, type /panel and select a **Warning Panel**.
11. Inside the panel, enter the executive trade-off rule:

Any stakeholder request to override the 20% Technical Debt allocation requires an executive waiver signed by the IT Director and a documented CMMI Process Deviation entry acknowledging that subsequent release delivery dates will slip by a factor of 1.5x due to defect drag.

12. Click **Publish** in the top-right corner.

Activity 2.2: Auditing Static Code Analysis Reports and Generating Debt Tickets in Jira

- **Tool:** Jira Software
- **Objective:** Translate technical code quality audit findings (e.g., high cyclomatic complexity, missing unit tests, unpatched dependencies) into prioritized, business-justified Jira technical debt tickets.

Step-by-Step Instructions:

1. Open Jira Software and go to your active project backlog.
2. Click + **Create** in the top navigation bar.
3. Configure **Technical Debt Ticket 1 (Maintainability & Complexity)**:
 - *Issue Type:* Select Task (or Technical Debt if custom configured).
 - *Summary:* [TECH DEBT] Refactor Payment Processor Interface to Reduce Cyclomatic Complexity
 - *Components:* Backend / Core Ledger
 - *Story Points:* 5
 - *Description:* Copy and paste this structured business justification:

```
### CMMI Technical Solution (TS) & Verification Audit:
- **Static Analysis Source:** SonarQube Report Module: `PaymentGatewayAdapter.java`
- **Current State:** Cyclomatic complexity score = 28 (Maximum threshold is 10). Class contains 6 nested conditional switches.
- **Risk / Defect Impact:** Any change to third-party payment endpoints incurs an 80% regression risk in core checkout.
- **Proposed Refactoring:** Implement Strategy Design Pattern to extract multi-currency routing into isolated, testable adapters.
- **Acceptance Criteria (DoD):**
  - [ ] Cyclomatic complexity reduced below 8.
  - [ ] Unit test branch coverage increased from 42% to >= 85%.
  - [ ] Zero functional regression across existing payment suites.
```

4. Click **Create**.
5. Click + **Create** again to configure **Technical Debt Ticket 2 (Security & Dependencies)**:
 - *Issue Type*: Task
 - *Summary*: [TECH DEBT] Upgrade Spring Boot Framework to Resolve High CVE Vulnerabilities
 - *Story Points*: 3
 - *Description*:

Security Compliance Audit:

- ****Vulnerability ID:**** CVE-2024-XXXX (Remote Code Execution vulnerability in outdated JSON parser library).
- ****Action:**** Upgrade core microservice framework baseline to latest LTS version.
- ****Acceptance Criteria:****
 - [] Framework version bumped to v3.2.x.
 - [] Snyk / OWASP dependency scan returns 0 High/Critical alerts.
 - [] CI pipeline integration smoke tests pass in staging.

6. Click **Create**.
7. Navigate to your **Backlog** view.
8. Locate your active sprint container. Note the current total planned story points.
9. Drag both newly created technical debt tasks ([TECH DEBT] Refactor Payment Processor Interface and [TECH DEBT] Upgrade Spring Boot Framework) directly into the active sprint.
10. Calculate the percentage: verify that these 8 points represent exactly **20%** of a standard 40-point sprint capacity, demonstrating a balanced, audit-ready sprint backlog.

Cross-Functional Team Collaboration: Individual Hands-on Lab Guide

1: Aligning Engineering, Design, and QA Resources

Activity 1.1: Designing a Dual-Track Agile & "Three Amigos" Hand-off Flow in Lucidchart

- **Tool**: Lucidchart (or Miro)
- **Objective**: Architect an end-to-end cross-functional collaboration model mapping out how UX Design, Engineering, and QA synchronize across Discovery and Delivery tracks to satisfy CMMI *Integrated Project Management (IPM)* and *Product Integration (PI)*.

Step-by-Step Instructions:

1. Log in to Lucidchart, click + **New** > **Lucidchart** > **Blank Document**, and title it: [Your Name] - Cross-Functional Dual-Track Collaboration Flow.
2. From the shape menu on the left, drag a **Horizontal Swimlane** container onto the canvas and set it to **3 swimlanes**:
 - Label Swimlane 1: Track 1: Product Discovery (Product Owner & UX/UI Designer)
 - Label Swimlane 2: Cross-Functional Alignment (The Three Amigos Gateway)

- Label Swimlane 3: Track 2: Product Delivery (Software Engineers & QA SDETs)
- 3. Set the background of Swimlane 1 to light purple, Swimlane 2 to light yellow, and Swimlane 3 to light blue.
- 4. In **Swimlane 1 (Discovery)**, create 2 sequential process blocks:
 - Block A: User Research & Interactive Figma Wireframing (Sprint N-1)
 - Block B: Drafting User Story & Business Intent (Confluence)
- 5. In **Swimlane 2 (Alignment)**, place a diamond (Decision Gate) centered horizontally between Discovery and Delivery:
 - Diamond Label: Three Amigos Alignment Session: Definition of Ready (DoR) Gate
- 6. Below the diamond, place a sticky note detailing the 3 mandatory perspectives:
 - Product/UX: "What problem are we solving and what does it look like?"
 - Engineering: "How technically feasible and scalable is this architecture?"
 - QA: "What edge cases, failure states, and test scenarios must we validate?"
- 7. In **Swimlane 3 (Delivery)**, create 3 sequential process blocks:
 - Block A: Sprint Execution: In-Sprint Feature Development (Sprint N)
 - Block B: Automated Integration Testing & QA Regression Passes
 - Block C: Increment Definition of Done (DoD) Sign-Off
- 8. Draw directional connection arrows showing the sequence:
 - Connect Block A in Discovery → Block B in Discovery → the **Three Amigos Gate** in Swimlane 2.
 - Draw two outcome arrows leaving the **Three Amigos Gate**:
 - Arrow 1 (Down / Green): Label DoR Criteria Met → points to Block A in Delivery (Sprint N).
 - Arrow 2 (Up-Loop / Red): Label DoR Failed (Missing Edge Cases / Infeasible) → loops back to Block A in Discovery.
 - Connect Delivery Block A → Block B → Block C.
- 9. Click **File > Export > PNG** (or copy the shareable document link) for submission.

Activity 1.2: Configuring a "Three Amigos" Review Subtask System in Jira

- **Tool:** Jira Software
- **Objective:** Embed mandatory cross-functional review subtasks and custom checklist fields into Jira User Stories, ensuring stories cannot enter active development without formal Design, Engineering, and QA clearance.

Step-by-Step Instructions:

1. Open Jira Software and navigate to your project backlog.
2. Click **+ Create** in the top navigation bar to create a feature story:
 - *Issue Type:* Story
 - *Summary:* Implement Biometric Touch/FaceID Mobile Authentication
 - *Story Points:* 5
 - *Description:* Allow mobile app users to authenticate sessions via device biometrics with secure token storage.
3. Click **Create**.
4. Open the newly created story from your backlog.

5. In the issue details view, click the **Add subtask** (or **+ Subtask**) button to generate 3 mandatory alignment verification items:
 - **Subtask 1:**
 - *Summary:* [Three Amigos] UX/Design Review: Figma Tokens & Error State Handoff
 - *Component:* Design
 - **Subtask 2:**
 - *Summary:* [Three Amigos] Dev Feasibility: Native Device Keychain Integration Review
 - *Component:* Architecture
 - **Subtask 3:**
 - *Summary:* [Three Amigos] QA Sign-Off: Biometric Hardware Mismatch & Lockout Scenarios
 - *Component:* Quality Assurance
6. Open **Subtask 3 (QA Sign-Off)** and add a comment:
 - QA Pre-Development Verification: Reviewed biometric failure edge cases. Edge cases added: (1) 5 failed biometric attempts must trigger PIN fallback; (2) OS-level biometric enrollment revocation must force re-authentication. Story ready for sprint.
7. Click **Save** on the comment.
8. Transition **Subtask 1**, **Subtask 2**, and **Subtask 3** to Done.
9. Add the label DoR-Validated to the parent User Story, demonstrating that all three cross-functional pillars approved the specification before engineering begins.

2: Facilitating Effective Communication Channels

Activity 2.1: Designing an Enterprise Communication Channel Matrix in Confluence

- **Tool:** Confluence
- **Objective:** Author an institutional Communication and Operating Agreement standard that delineates synchronous vs. asynchronous channels, defines notification etiquettes, and maintains auditable CMMI decision records.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - Cross-Functional Communication & Channel Architecture Policy.
3. Add a brief executive statement using /heading 2: 1. Purpose & Core Principles.
4. Insert text: To eliminate communication fragmentation, protect engineering deep-work focus, and guarantee audit-ready governance trails, all project communications must adhere to this structured channel routing standard.
5. Type /table and press **Enter** to insert a 5-column, 5-row table.
6. Populate the header row:
 - Channel / Platform
 - Communication Category
 - Target Audience
 - Permitted Communication Scope
 - Response Time Expectation (SLA)

7. Populate the table rows with enterprise communication guidelines:

- **Row 1:**
 - *Channel:* MS Teams: #proj-announcements
 - *Category:* Asynchronous (Broadcast)
 - *Audience:* All Project Stakeholders
 - *Scope:* Milestone completions, release schedules, stage-gate outcomes, corporate alerts. No conversational threads.
 - *SLA:* Read within 24 hours.
- **Row 2:**
 - *Channel:* MS Teams: #dev-qa-coordination
 - *Category:* Asynchronous (Tactical)
 - *Audience:* Developers, SDETs, DevOps
 - *Scope:* Environment availability, staging deployment alerts, pull request review requests.
 - *SLA:* Respond within 4 hours during business hours.
- **Row 3:**
 - *Channel:* MS Teams: #incident-war-room (Sync)
 - *Category:* Synchronous (Emergency)
 - *Audience:* Incident Commander, On-Call Devs, Tech Lead
 - *Scope:* Critical Sev 1 production outages, immediate blocker escalations. Voice/video bridges.
 - *SLA:* Immediate (<15 minutes response).
- **Row 4:**
 - *Channel:* Jira Ticket Comments & Confluence Pages
 - *Category:* Institutional Audit Record
 - *Audience:* Engineering, Product, PMO, CMMI Auditors
 - *Scope:* Scope decisions, architectural trade-offs (DAR), requirements clarification, defect triage.
 - *SLA:* Acknowledged during daily stand-up / refinement.

8. Below the table, insert an **Info Panel** (/panel) titled: Golden Communication Rule:

- Text: Substantive technical decisions, scope trade-offs, and approval sign-offs made over MS Teams voice or chat must be summarized and linked to a Jira issue or Confluence decision log within 2 hours. Chat logs are ephemeral; CMMI governance demands persistent records.

9. Click **Publish** in the top-right corner.

Activity 2.2: Building a Structured Escalation and Communication Hub in MS Teams

- **Tool:** MS Teams
- **Objective:** Construct an operational MS Teams channel hub equipped with pinned escalation runbooks, automated connector announcements, and structured triage tags to streamline cross-team communications.

Step-by-Step Instructions:

1. Open MS Teams and navigate to your team workspace (or create a sandbox team: [Your Name] - Enterprise Project Hub).
2. Click the three dots (...) next to the team name and select **Add channel**.
3. Set the Channel name to: blocker-escalations.
4. Set Channel type to **Standard** and click **Create**.
5. Inside the #blocker-escalations channel, click **New post** (or **Start a post**).
6. Click the **Format** icon (letter A with pen) to open the rich-text editor and select **Announcement** from the dropdown menu.
7. Set the Headline banner to: CROSS-FUNCTIONAL BLOCKER ESCALATION RUNBOOK.
8. In the body of the announcement, copy and paste this standard escalation matrix:

****When to Post in this Channel:****

Use this channel **ONLY** when an impediment blocks active sprint progress and has remained unresolved for >2 hours.

****Escalation Template (Copy & Fill):****

- ****Jira Issue Key:**** [e.g., PAY-204]
- ****Blocked Function / Role:**** [e.g., Front-End Engineering]
- ****Blocking Entity / Dependency:**** [e.g., Core Banking Sandbox API 500 error]
- ****Hours Blocked:**** [e.g., 3.5 hours]
- ****Direct Business Impact:**** [e.g., Blocks completion of Sprint Goal user story]
- ****Tag Assigned PM:**** [@ModernPM]

9. Click **Post**.
10. Click the three dots (...) on your newly published announcement post and select **Pin**. Confirm by clicking **Pin for everyone**.
11. Click the + (Add a tab) button at the top of the channel navigation bar.
12. Search for and select **Website** (or **Jira Cloud** / **Confluence Cloud** if available in your workspace).
13. Set the Tab Name to Active Jira Sprint Board and paste your project board URL. Click **Save**.
14. Verify that the channel provides both the communication runbook and direct one-click access to the delivery board.

3: Fostering a Culture of Accountability and Psychological Safety

Activity 3.1: Conducting a Blameless Post-Mortem and Root-Cause Mapping in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Run a structured, blameless post-mortem using a 5-Whys and Fishbone (Ishikawa) diagram to analyze an escaped production bug, shifting focus from individual blame to systemic process failure (satisfying CMMI *Causal Analysis and Resolution - CAR*).

Step-by-Step Instructions:

1. Open Miro, click **+ New board > Create a blank board**, and title it: [Your Name] - Blameless Post-Mortem & CAR Investigation.
2. Add a prominent header across the top using the **Text** tool (T): Blameless Incident Investigation: Sev 1 Payment Double-Billing Incident.
3. In the upper-left corner, create a green sticky note titled Psychological Safety Baseline:
 - Text: We operate under the fundamental premise: Engineers work with competence and good intent. Failures are caused by systemic gaps, inadequate automated guardrails, or ambiguous processes, never individual incompetence.
4. Draw a horizontal central line across the canvas representing the spine of a **Fishbone Diagram**, terminating at a red box labeled: Defect: 140 Customers Charged Twice During Morning Peak Hours.
5. Draw 4 diagonal rib lines branching off the central spine, labeling the process categories:
 - Rib 1 (Top Left): Process & Requirements
 - Rib 2 (Bottom Left): Environment & Configuration
 - Rib 3 (Top Right): Testing & Verification
 - Rib 4 (Bottom Right): Architecture & Code Safety
6. Attach sticky notes to each rib detailing systemic contributing factors:
 - **On Rib 1 (Process):** Sticky (Yellow) Story acceptance criteria did not explicitly mandate idempotency token validation on the payment gateway adapter.
 - **On Rib 2 (Environment):** Sticky (Yellow) Staging database did not simulate network packet drops or slow upstream socket responses.
 - **On Rib 3 (Testing):** Sticky (Yellow) Automated regression suite had 0 concurrent load tests covering double-click button submissions.
 - **On Rib 4 (Architecture):** Sticky (Yellow) Client-side submit button remained active during asynchronous API request transmission.
7. Below the Fishbone diagram, create a container using the **Shape** tool titled: CMMI CAR Systemic Corrective Action Register.
8. Place 2 blue sticky notes inside the register defining process improvements:
 - *Action 1:* Update Organizational Definition of Ready (DoR): All transactional API stories must include explicit Idempotency Key validation criteria.
 - *Action 2:* CI/CD Pipeline Guardrail: Embed a Gatling concurrent-load test gate that simulates 1,000 rapid duplicate transactions prior to production release.
9. Draw connector lines from the yellow root-cause stickies directly to the blue corrective actions, proving an unbroken link between systemic analysis and permanent process remediation.

Activity 3.2: Formalizing a Team Working Agreement and Safety Compact in Confluence

- **Tool:** Confluence
- **Objective:** Author a formal, collaborative Team Working Agreement that sets operational accountability baselines, protects code review psychological safety, and satisfies CMMI *Integrated Project Management (IPM)* governance expectations.

Step-by-Step Instructions:

1. In your Confluence project space, click **Create** (+).
2. Title the page: [Your Name] - Team Working Agreement & Psychological Safety Compact.
3. Use /heading 2 to add: 1. Team Norms & Operating Baseline.
4. Type /table and insert a 3-column, 4-row table detailing team operating commitments:
 - Domain | Operational Standard | Accountability Mechanism
 - Core Collaboration Hours | 10:00 AM - 03:00 PM local time for synchronous pairing and stand-ups. | Slack/Teams calendar status updated accurately.
 - Code Review Protocol | Every PR reviewed within 4 business hours. Feedback must be constructive and rationale-based. | GitHub/Bitbucket PR reminders configured.
 - Stand-up Cadence | Strictly 15 minutes. Focus on flow blockers, not micro-status reporting. | PM moves deep technical debates to parking lot.
5. Use /heading 2 to add: 2. Code Review Safety & Constructive Feedback Guidelines.
6. Insert a 2-column table contrasting constructive vs. toxic review behaviors:
 - Unacceptable / Toxic Feedback (Avoid) | Psychologically Safe & Constructive Feedback (Adopt)
 - *"Why did you write this terrible nested loop?"* | *"Could we extract this logic into a helper method to reduce cyclomatic complexity and improve readability?"*
 - *"This code makes no sense."* | *"I'm having trouble following this state transition; could we walk through this logic together or add a brief explanatory comment?"*
 - *"This is wrong. Rewrite it."* | *"According to our coding standard (PAL-STD-003), we prefer immutability here. What do you think about using a builder pattern?"*
7. Below the tables, insert an **Info Panel** (/panel) titled: Blameless Escalation Standard:
 - Text: Team members are expected and encouraged to surface risks, missing dependencies, or inadvertent mistakes immediately without fear of reprisal. Catching an error early is celebrated as a team cost-saving win under CMMI Verification standards.
8. Click **Publish** in the top-right corner.

4: Managing Stakeholder Expectations and Feedback Loops

Activity 4.1: Building a Stakeholder Power/Interest Matrix & Cadence Plan in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Map project stakeholders onto a Power vs. Interest grid, defining customized communication rhythms and formal feedback loops to maintain alignment and satisfy CMMI *Project Planning (PP)* stakeholder involvement practices.

Step-by-Step Instructions:

1. Open Lucidchart, click **+ New > Lucidchart > Blank Document**, and title it: [Your Name] - Stakeholder Power-Interest Matrix & Feedback Cadence.
2. Draw a large square on the canvas and divide it into 4 equal quadrants using perpendicular horizontal and vertical lines.
3. Label the axes:
 - **Vertical Axis (Y):** Stakeholder Power / Organizational Influence (Low -> High)
 - **Horizontal Axis (X):** Stakeholder Interest in SDLC Outcomes (Low -> High)
4. Label each quadrant:
 - **Top-Right (High Power / High Interest):** Manage Closely (Collaborate & Partner)
 - **Top-Left (High Power / Low Interest):** Keep Satisfied (Strategic Governance)
 - **Bottom-Right (Low Power / High Interest):** Keep Informed (Frequent Feedback Loops)
 - **Bottom-Left (Low Power / Low Interest):** Monitor (Minimal Effort)
5. Color the Top-Right quadrant light green, Top-Left light yellow, Bottom-Right light blue, and Bottom-Left light gray.
6. Drag rectangular cards representing distinct project stakeholders into their appropriate quadrants:
 - **In Top-Right (Manage Closely):**
 - Card 1: Chief Product Officer (CPO) & Executive Sponsor
 - Card 2: Lead Systems Architect & Security CISO
 - **In Top-Left (Keep Satisfied):**
 - Card 3: Chief Financial Officer (Budget Approver)
 - Card 4: Internal Legal & Regulatory Compliance Officer
 - **In Bottom-Right (Keep Informed):**
 - Card 5: Customer Support Lead & Operations Specialists
 - Card 6: End-User Advisory Group (Beta Testers)
 - **In Bottom-Left (Monitor):**
 - Card 7: Third-Party Vendor Software Sales Rep
7. Below the matrix, create a 3-column table detailing the **Mandatory Feedback Cadence**:
 - Column 1: Quadrant Category | Column 2: Engagement Mechanism | Column 3: Frequency
 - Manage Closely | Sprint Demos & Bi-Weekly Executive Scorecard | Every 2 Weeks
 - Keep Satisfied | Quarterly Budget/Milestone Review & Gate Audits | Monthly / Per Phase
 - Keep Informed | Release Notes Newsletter & Async Jira Portal | Weekly / Per Sprint
 - Monitor | Published Release Roadmap Updates | Quarterly
8. Export the diagram as a PDF or PNG for submission.

Activity 4.2: Creating an Executive Sprint Review & Feedback Digest in Confluence

- **Tool:** Confluence
- **Objective:** Author an executive-ready Sprint Review summary that translates technical development telemetry into business-value outcomes, captures stakeholder feedback, and formally logs scope trade-offs.

Step-by-Step Instructions:

1. Open your Confluence project space and click **Create** (+).
2. Title the page: [Your Name] - Sprint 12 Executive Review & Stakeholder Feedback Digest.
3. Use /heading 2 to add: 1. Executive Summary & Sprint Goal Achievement.
4. Type /table and insert a 2-column, 4-row status summary:
 - Sprint Metric | Actual Delivery Status
 - Sprint 12 Goal | Deploy Real-Time Multi-Currency Forex Quoting Engine to Staging.
 - Goal Commitment Status | **ACHIEVED (100% of core journey functional)**
 - Story Points Delivered | 36 Delivered / 38 Committed (94.7% Commitment Reliability)
 - Escaped Defect Rate | 0 Critical / 1 Minor (Sev 4 UI alignment)
5. Use /heading 2 to add: 2. Demonstrated Business Capabilities (Working Software).
6. Insert 3 bullet points describing business value delivered (not code tasks):
 - *Institutional traders can now stream live EUR/USD, GBP/USD, and JPY/USD spot quotes with sub-20ms latency.*
 - *Integrated automated 60-second rate-lock countdown preventing execution slippage.*
 - *Automated compliance audit log recording all trade rejections directly into an immutable database.*
7. Use /heading 2 to add: 3. Stakeholder Feedback & Trade-Off Decisions.
8. Type /table and insert a 4-column, 3-row structured feedback log:
 - Stakeholder Name & Role | Feedback / Feature Request Raised | Agreed Governance Action | Impact on Baseline
 - VP of Institutional Sales | *"We need support for AUD and CAD currency pairs added immediately in Sprint 13."* | Approved via Scope-Swap: Swapping out 5-point low-priority CSV export story. | Net capacity neutral (5 pts swapped).
 - Head of Information Security | *"Audit logs must include client IP addresses to satisfy regional banking regulations."* | Accepted as mandatory CMMI compliance requirement; added to Sprint 13 backlog. | 3 points absorbed into technical buffer.
9. Below the table, insert a **Success Panel** (/panel):
 - Heading: Formal Stakeholder Sign-Off Baseline:
 - Text: Sprint 12 deliverables accepted as demonstrated. Sprint 13 scope baseline finalized and frozen following approved scope-swap adjustments.
10. Click **Publish** in the top-right corner.

Release Planning and Delivery (CI/CD Overview): Individual Hands-on Lab Guide

1: Understanding Continuous Integration and Deployment Flows

Activity 1.1: Designing an Enterprise CI/CD Pipeline Architecture in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Design an automated CI/CD pipeline architecture detailing every validation gate from developer code commit to production deployment, explicitly mapping where CMMI *Verification (VER)* and *Configuration Management (CM)* audit records are created.

Step-by-Step Instructions:

1. Open Lucidchart, click + **New** > **Lucidchart** > **Blank Document**, and title the document: [Your Name] - Enterprise CI/CD Pipeline Architecture & Governance Gates.
2. From the left shape panel, drag a **Horizontal Swimlane** container onto the canvas and configure it with **4 lanes**:
 - Label Swimlane 1: Continuous Integration (Build & Code Hygiene)
 - Label Swimlane 2: Automated Verification (Static & Dynamic Testing)
 - Label Swimlane 3: Staging Deployment & Validation (UAT Environment)
 - Label Swimlane 4: Production Release & Telemetry (Zero-Downtime CD)
3. Color-code the lanes: Swimlane 1 light blue, Swimlane 2 light purple, Swimlane 3 light yellow, and Swimlane 4 light green.
4. In **Swimlane 1 (CI)**, drag and connect 3 sequential process rectangles:
 - Block A: Developer Git Push (Trunk / Short-Lived Feature Branch)
 - Block B: Automated Pipeline Trigger & Dependency Compilation
 - Block C: Unit Test Execution (Target: >=80% Code Coverage)
5. In **Swimlane 2 (Automated Verification)**, place 2 process blocks and a decision diamond:
 - Block A: Static Code Analysis (SonarQube Gate: 0 Blocker/Critical Bugs)
 - Block B: Container Vulnerability & Dependency Scan (Snyk / Trivy)
 - Diamond: Automated CI Gate: Pass All Scans & Unit Tests?
6. In **Swimlane 3 (Staging & Validation)**, place 3 process blocks:
 - Block A: Immutable Container Image Built & Pushed to Registry (Nexus/Artifactory)
 - Block B: Automated Deployment to Staging Kubernetes Cluster
 - Block C: Automated API Integration Suite & End-to-End Regression Pass
7. In **Swimlane 4 (Production CD)**, place 2 process blocks and an operational monitor:
 - Block A: Production Canary Rollout (5% Initial Traffic Routing)
 - Block B: Traffic Scaling to 100% Following SLA Health Check
 - Block C (Cylinder/Storage): Automated Audit Log Generated (CMMI VER/CM Evidence)
8. Draw directional connection lines:
 - Connect all blocks sequentially across the swimlanes from Swimlane 1 to Swimlane 4.
 - On the Decision Diamond in Swimlane 2, draw a red arrow from the "No / Fail" exit looping back to Block A in Swimlane 1 with the label Pipeline Failed: Build Aborted & Jira Alert Fired.

9. Add callout bubbles highlighting the CMMI audit evidence points:
 - Callout on Swimlane 2 Diamond: CMMI VER: Automated peer-quality verification without manual intervention.
 - Callout on Swimlane 3 Block A: CMMI CM: Cryptographic SHA-256 binary hashing for baseline integrity.
10. Click **File > Export > PNG** (or copy the share link) for submission.

Activity 1.2: Automating Jira Ticket Transitions via Simulated CI/CD Build Triggers

- **Tool:** Jira Software
- **Objective:** Configure Jira Automation rules that simulate CI/CD webhook events to transition Jira User Stories automatically as code builds and passes pipeline test gates, eliminating manual status updating.

Step-by-Step Instructions:

1. In Jira Software, navigate to your project and click **Project settings** (gear icon) > **Automation**.
2. Click **Create rule** in the top-right corner.
3. In the trigger search bar, type Incoming webhook and select it.
4. Copy the unique **Webhook URL** provided by Jira (this URL simulates the webhook your Jenkins, GitHub Actions, or GitLab CI runner calls when a build passes).
5. Set the **Issue filter** selector to: Issue key from webhook data (or use JQL: `issueType = Story AND status = "In Development"`). Click **Save**.
6. Click **+ New action**, then select **Transition issue**.
7. In the **To status** dropdown, select In Testing (or Ready for QA). Click **Save**.
8. Click **+ New action**, then select **Add comment**.
9. In the comment body field, copy and paste this automated build log entry:

```
**AUTOMATED CI/CD BUILD NOTIFICATION**
- **Pipeline Job:** `pipeline-build-core-banking-api`
- **Build Number:** `#1042`
- **Unit Test Result:** `248 Passed / 0 Failed` (Coverage: 84.6%)
- **SonarQube Quality Gate:** `PASSED` (0 Critical Security Vulnerabilities)
- **Artifact SHA-256:** `e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855`
- **Status:** Automatically transitioned to Verification stage.
```

10. Click **Save**.
11. Name the rule: [Your Name] - Automated CI Build Transition & Audit Logger.
12. Click **Turn on rule**.
13. Manually simulate the rule execution:
 - Go to your active board and locate a User Story in In Development.
 - Open the issue, click the **Actions (...)** menu, select **Automation**, locate your newly configured rule, and click **Run rule**.
 - Verify that the ticket moves to In Testing and that the formatted build verification comment appears in the issue activity stream.

2: Managing Build Artifacts, Versions, and Release Branches

Activity 2.1: Designing a GitFlow & Release Branching Architecture in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Construct an enterprise branching and versioning blueprint mapping Git feature branches, release freeze lines, semantic versioning tags, and hotfix paths to satisfy CMMI *Configuration Management (CM)* baseline controls.

Step-by-Step Instructions:

1. Open Miro, click **+ New board > Create a blank board**, and title it: [Your Name] - GitFlow Release Branching & SemVer Architecture.
2. Add a title banner at the top of the canvas: Enterprise GitFlow Branching Model & Release Baselines.
3. Draw 4 horizontal parallel lines spanning across the canvas to represent the core branch tracks:
 - Track 1 (Top - Dark Blue): main (Production Deployable Baseline)
 - Track 2 (Second - Orange): hotfix/* (Critical Production Fixes)
 - Track 3 (Third - Purple): release/* (Release Candidates & Freeze Lines)
 - Track 4 (Bottom - Green): develop (Active Sprint Integration Mainline)
4. Add small circular nodes (O) along the **develop** track representing sprint commits:
 - Node 1: Commit A (Story PAY-101)
 - Node 2: Commit B (Story PAY-102)
5. Branch off from develop: Draw an upward diagonal arrow from Node 2 to Track 3 (**release/***) to create a release candidate node:
 - Label: Branch: release/v2.1.0-RC1 (Feature Freeze Milestone)
6. Add 2 hardening commits along the release/v2.1.0-RC1 line representing bug fixes during staging verification:
 - Label: Bugfix Commit RC1.1 (Defect DEF-401)
7. Draw a split merge arrow out of the final release hardening node:
 - Arrow A (Upward to main): Terminates in a production deployment node labeled TAG: v2.1.0 (MAJOR.MINOR.PATCH Production Baseline).
 - Arrow B (Downward back to develop): Merges the bugfix back into the develop track to prevent regression.
8. Model a production emergency on Track 2 (**hotfix/***):
 - Draw a branch emerging from the v2.1.0 tag on main downward to hotfix/*.
 - Label the node: Branch: hotfix/v2.1.1 (CVE Security Patch).
 - Draw dual merge-back arrows: one merging into main tagged as TAG: v2.1.1, and one merging into develop.
9. Below the diagram, add a sticky note table summarizing **Semantic Versioning Rules**:
 - MAJOR (v3.0.0): Incompatible API / database schema breaking changes.
 - MINOR (v2.2.0): Backward-compatible new functional features added.
 - PATCH (v2.1.1): Backward-compatible bug fixes and security hotfixes.
10. Export the board as an image or generate a shareable view link for submission.

Activity 2.2: Creating an Immutable Release Baseline and Runbook in Confluence

- **Tool:** Confluence
- **Objective:** Author a formal CMMI Level 3 Release Configuration Baseline in Confluence that links Jira release versions, immutable artifact checksums, Git commit hashes, and rollback procedures.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create** (+).
2. Title the page: [Your Name] - Release Baseline & Deployment Runbook: v2.1.0.
3. Use /heading 2 to add: 1. Release Identification & Configuration Audit Baseline (CMMI CM).
4. Type /table and insert a 2-column, 6-row metadata table:
 - Configuration Parameter | Audit Value
 - Release Version (SemVer) | v2.1.0
 - Target Deployment Date | October 20, 2026 - 22:00 UTC
 - Git Commit Hash (SHA-1) | a4f8c29d10e5871b63e8a939f02931a74d2843b1
 - Docker Container Image URI | registry.corp.internal/payments/engine:v2.1.0
 - Artifact Cryptographic Digest | SHA-256:
7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284add200126d9069
 - Jira Fix Version Link | Release 1.0 - Core Banking API
5. Use /heading 2 to add: 2. Pre-Deployment Stage-Gate Checklist.
6. Type /task-list and add 4 mandatory verification checks:
 - [] 1. All stories in Jira release version marked Done with zero unresolved Sev 1/2 bugs.
 - [] 2. Database schema rollback migration script validated on staging instance.
 - [] 3. Cloud infrastructure capacity scaling buffer verified (>30% headroom).
 - [] 4. Incident war room voice bridge established on MS Teams.
7. Use /heading 2 to add: 3. Step-by-Step Production Cutover & Rollback Runbook.
8. Insert an ordered sequence of technical actions:
 - Step 1 (T-00:15): Put active session router into maintenance read-only mode.
 - Step 2 (T-00:00): Execute flyway database schema upgrade script.
 - Step 3 (T+00:10): Deploy Kubernetes pods running artifact v2.1.0.
 - Step 4 (T+00:20): Run automated synthetic smoke test suite against live health endpoint.
9. Below the sequence, insert an **Alert / Warning Panel** (/panel) titled: Automated Rollback Trigger:
 - Text: If synthetic smoke tests fail or HTTP 5xx error rates exceed 1.0% for 3 consecutive minutes, abort release immediately: trigger script 'rollback-v2.0.9.sh', restore database snapshot, and notify Incident Commander.
10. Click **Publish** in the top-right corner.

3: Coordinating Blue-Green, Beta, and Canary Deployments

Activity 3.1: Constructing a Blue-Green vs. Canary Deployment Trade-Off Matrix in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Design an interactive decision matrix and architectural routing flow comparing Blue-Green, Canary, and Feature-Flagged Beta deployments to facilitate CMMI *Decision Analysis and Resolution (DAR)* for release governance.

Step-by-Step Instructions:

1. Open Miro and click **+ New board > Create a blank board**.
2. Title the board: [Your Name] - Deployment Strategy Architecture & DAR Matrix.
3. In the top half of the canvas, draw two parallel system routing diagrams using the **Shape** tool (R):
 - **Diagram A (Blue-Green Routing):**
 - Draw a box labeled Router / Load Balancer.
 - Draw two downstream environment boxes: Environment Blue (Active Live v1.0 - 100% Traffic) and Environment Green (Idle Staging v2.0 - 0% Traffic).
 - Draw a thick switch arrow showing the router flipping traffic entirely from Blue to Green.
 - **Diagram B (Canary Progressive Routing):**
 - Draw a box labeled Smart Traffic Splitter / Service Mesh.
 - Draw two downstream target boxes: Stable Baseline v1.0 (95% Production Traffic) and Canary Pod v2.0 (5% Production Traffic).
 - Add an observability feedback arrow from the Canary Pod back to the Splitter labeled Prometheus Latency & Error Telemetry: Auto-Scale or Auto-Rollback.
4. In the bottom half of the canvas, insert a 5-column, 4-row **Table** representing the **CMMI DAR Strategy Matrix**:
 - Headers: Deployment Pattern | Infrastructure Cost | Rollback Latency | Stateful Database Complexity | Blast Radius Risk
 - Row 1: Blue-Green Deployment | High (Requires 2x identical hardware/cloud clusters) | Near Instant (<30 seconds via router flip) | High (Requires backward-compatible database schemas) | Medium (All users transition simultaneously)
 - Row 2: Canary Deployment | Low-Medium (Requires minimal additional pods) | Fast (<2 minutes to divert traffic) | Medium (Requires backward-compatible schemas) | Extremely Low (Confined to 5% sample traffic)
 - Row 3: Feature-Flagged Beta | Low (Runs within existing production instances) | Instantaneous (<5 seconds via runtime flag toggle) | Low-Medium (Flags isolate conditional execution) | Zero (Confined strictly to opt-in beta personas)
5. Below the matrix, add a green sticky note (N):
 - Text: DAR Recommendation: For the Core Banking API release, Canary Deployment is selected. Its small blast radius protects financial transactions, while infrastructure costs remain 40% lower than a full duplicated Blue-Green environment.
6. Export the board as an image or copy the shareable link for submission.

Activity 3.2: Facilitating a Simulated Canary Outage & Rollback War Room in MS Teams

- **Tool:** MS Teams
- **Objective:** Coordinate a live, fast-paced deployment incident war room in MS Teams, interpreting simulated production error telemetry, issuing an abort/rollback decision, and recording CMMI release audit logs.

Step-by-Step Instructions:

1. Open MS Teams, navigate to your project team, and click the three dots (...) > **Add channel**.
2. Set the Channel name to: release-war-room-live.
3. Set the Channel privacy to **Standard** and click **Create**.
4. In the channel posts stream, click **New post**, expand the formatting editor, and select **Announcement**.
5. Set the Headline banner to: LIVE DEPLOYMENT: RELEASE v2.1.0 CANARY ROLLOUT (5% TRAFFIC).
6. Enter the initial release deployment broadcast:

****CANARY COMMENCEMENT LOG:****

- ****Target Version:**** `v2.1.0`
- ****Deployment Window:**** Active Now
- ****Traffic Distribution:**** 95% on v2.0.9 (Baseline) | 5% on v2.1.0 (Canary)
- ****Monitoring Telemetry:**** Datadog / Prometheus monitoring active on endpoint `/api/v1/settlements`.

7. Click **Post**.
8. Click **Reply** under your announcement to simulate incoming telemetry alerts:
 - Reply 1 (Simulated DevOps Bot): ALERT [T+04 min]: HTTP 500 Internal Server Errors on Canary pods increased to 4.2% (Threshold: 1.0%). P99 Latency spiked from 180ms to 1,240ms.
9. Click **Reply** again to simulate an on-call engineer report:
 - Reply 2 (Simulated Lead SDET): INVESTIGATION: Canary instances are encountering unhandled NullPointerExceptions during database connection pool exhaustion on concurrent settlement writes.
10. Click **Reply** and draft the **Project Manager / Incident Commander Intervention:**

****INCIDENT COMMANDER DIRECTIVE - ABORT & ROLLBACK:****

1. ****Decision:**** Canary deployment threshold breached. Under CMMI Risk Management & Deployment Runbook protocol, release v2.1.0 is ABORTED.
2. ****Immediate Action:**** DevOps to set Canary traffic weight to 0% immediately; divert 100% of user traffic to stable v2.0.9 baseline.
3. ****Containment:**** Confirm blast radius was restricted strictly to the 5% sample cohort.
4. ****Governance:**** Log Jira Incident ticket Sev 1 and schedule mandatory Post-Incident Review (PIR / CAR) within 24 hours.

11. Click the three dots (...) on your PM Directive reply, select **Pin**, and confirm the pin.
12. Verify that the war room thread preserves a clear, chronological timestamp record of the incident, containment decision, and rollback execution to satisfy CMMI Level 3 Process and Product Quality Assurance (PPQA) audit standards.

Quality Assurance & UAT: Individual Hands-on Lab Guide

1: Integrating QA Early to Reduce Defect Resolution Costs

Activity 1.1: Modeling the Defect Cost-Escalation Curve and Shift-Left Interventions in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Construct a comparative visual model demonstrating the exponential cost of defect resolution across SDLC phases (Boehm's Cost of Quality curve) and establish concrete shift-left QA intervention checkpoints to satisfy CMMI *Verification (VER)* practices.

Step-by-Step Instructions:

1. Open Miro, click **+ New board** > **Create a blank board**, and title it: [Your Name] - Shift-Left Cost-Escalation Model & QA Interventions.
2. Select the **Shape** tool (R) and draw a horizontal axis labeled SDLC Phase (Progression ->) and a vertical axis labeled Relative Cost to Fix Defect (\$).
3. Create 5 distinct stage stations along the horizontal axis using rounded rectangles:
 - Station 1: Requirements & Discovery
 - Station 2: Architecture & System Design
 - Station 3: In-Sprint Coding & Dev Unit Testing
 - Station 4: Staging & System Integration Testing (SIT)
 - Station 5: Production Operations (Post-Release)
4. Draw an exponential curve line (using the **Pen** or curved connector line) starting low at Station 1 and spiking steeply upward at Station 5.
5. Place a red price tag sticky note (N) above each phase to illustrate cost-to-fix escalation:
 - Above Station 1: \$100 (Baseline discovery cost)
 - Above Station 2: \$500 (Design rework)
 - Above Station 3: \$1,500 (Code refactor & re-test)
 - Above Station 4: \$5,000 (Regression re-testing, staging re-deployment)
 - Above Station 5: \$50,000+ (Emergency hotfix, data patching, reputational loss, SLA breach)
6. Below each of Stations 1, 2, and 3, add a green rectangular container titled Shift-Left QA Preventive Interventions:
 - **Under Station 1 (Requirements):** Place a green sticky note: Action: QA conducts Ambiguity Reviews & Gherkin Example Mapping during backlog grooming before story pointing.
 - **Under Station 2 (Architecture):** Place a green sticky note: Action: QA leads Testability Audits, ensuring microservice APIs provide mockable endpoints and logging hooks.
 - **Under Station 3 (Development):** Place a green sticky note: Action: QA pairs with developers on automated unit/component tests and enforces pre-commit static analysis lint gates.
7. Connect each green preventive action to its corresponding red cost note with a dashed line labeled Defect Deflected (Saves 90% Cost).
8. Click **Export this board** > **Save as image (PNG)** or generate a share link for submission.

Activity 1.2: Automating Shift-Left QA Acceptance Sign-Offs in Jira

- **Tool:** Jira Software
- **Objective:** Configure a Jira workflow transition that mandates a formal QA test specification review before an engineering issue can be marked "Ready for Development," enforcing early defect prevention directly inside sprint boards.

Step-by-Step Instructions:

1. Open your project in Jira Software and click **Project settings** (gear icon) > **Workflows**.
2. Click the **Pencil (Edit)** icon next to your primary story workflow to enter the workflow diagram designer.
3. Locate the transition arrow from Backlog (or Refinement) to Selected for Development (or Ready for Dev).
4. Click directly on that transition arrow to select it.
5. In the right-hand properties sidebar, click **Validators**, then click **Add validator**.
6. Select **Field Required Validator** and click **Add**.
7. In the required fields list, select QA Reviewer (or Assignee / custom field Test Scenarios Documented).
8. In the **Error message** field, type: CMMI VERIFICATION RULE: User story cannot enter Ready for Development without an assigned QA Lead and documented automated test scenarios.
9. Click **Add**.
10. Click **Publish Draft** in the top yellow workflow banner (select No to saving a backup).
11. Return to your active board or backlog and click **+ Create**:
 - *Issue Type:* Story
 - *Summary:* Implement Automated FX Rate Streaming Webhook
 - *Story Points:* 5
 - *Description:* Provide downstream clients with real-time rate changes via webhook callbacks.
12. Click **Create**.
13. Open the newly created story. Without entering any QA review notes or assigning a QA reviewer, attempt to drag the ticket into Ready for Development.
14. Confirm that Jira halts the transition and displays your configured CMMI verification error message.
15. Add a comment: QA Shift-Left Check: Test specifications drafted covering network dropouts, SSL handshake failures, and invalid payload formats. Verified ready for sprint.
16. Assign your profile to the designated review field, re-attempt the status transition, and verify successful entry into the development queue.

2: Managing the User Acceptance Testing (UAT) Phase

Activity 2.1: Designing an End-to-End Business Journey UAT Plan in Confluence

- **Tool:** Confluence
- **Objective:** Author a comprehensive, audit-ready User Acceptance Testing (UAT) Strategy and Test Execution Plan in Confluence, defining business user journeys, entrance/exit criteria, and formal CMMI *Validation (VAL)* sign-off baselines.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - Enterprise UAT Master Strategy & Validation Plan (CMMI VAL).
3. Use /heading 2 to add: 1. UAT Objectives & Operational Scope.
4. Add text: This plan governs business validation for Release 1.0. While System Integration Testing (SIT) verified technical code specifications, UAT validates that the end-to-end platform satisfies real-world accounting workflows and regulatory mandates.
5. Use /heading 2 to add: 2. Strict UAT Entrance and Exit Criteria.
6. Type /table and insert a 3-column, 5-row governance criteria table:
 - Stage Gate | Mandatory Condition | Verification Method
 - UAT Entrance Gate | 100% SIT test cases executed with $\geq 98\%$ pass rate. | Jira SIT Test Execution Report
 - UAT Entrance Gate | Zero open Severity 1 (Blocker) or Severity 2 (Critical) technical bugs. | Jira Defect Dashboard
 - UAT Entrance Gate | Staging environment provisioned with sanitized production-like data. | DevOps Staging Sign-Off
 - UAT Exit Gate | 100% of defined core business validation scenarios executed. | UAT Execution Log
 - UAT Exit Gate | Formal written sign-off from designated Business Product Owner. | Confluence Electronic Sign-Off
7. Use /heading 2 to add: 3. End-to-End Business User Test Scenarios.
8. Type /table and insert a 5-column, 3-row test scenario register:
 - Scenario ID | Business User Persona | End-to-End Operational Journey | Expected Business Outcome | Pass / Fail Status
 - UAT-JOURNEY-01 | Treasury Analyst | Initiate \$2.5M USD corporate wire transfer, trigger two-factor approval, and download PDF audit receipt. | Ledger balance debited accurately, wire executed in settlement mock, compliance record generated. | PASS
 - UAT-JOURNEY-02 | Compliance Officer | Simulate an offshore account transaction flagged on the OFAC watch list. | Transaction suspended instantly, notification dispatched to compliance officer, audit reason recorded. | PASS
9. Below the table, insert a **Success Panel** (/panel) titled: CMMI VAL Stakeholder Sign-Off Register:
 - Text: By electronically signing below, the Business Lead confirms that the software is accepted for production release and satisfies all operational business drivers.
 - Include signature placeholders: Business Product Owner: [Approved] | Modern Project Manager: [Approved].
10. Click **Publish** in the top-right corner.

Activity 2.2: Building a Dedicated UAT Validation Board and Execution Run in Jira

- **Tool:** Jira Software
- **Objective:** Configure a dedicated UAT Kanban tracking board in Jira with custom issue types, business user execution statuses, and automated UAT completion metrics.

Step-by-Step Instructions:

1. In Jira Software, navigate to your project and click **Project settings > Issue types**.
2. Click **Add issue type**, name it UAT Test Case, select a checklist or clipboard icon, and click **Create**.
3. Return to your Jira project, click the project dropdown in the top bar, and select **Create project > Kanban > Company-managed** (or create a new board inside your existing project).
4. Name the board: [Your Name] - Business UAT Execution Hub.
5. On the board, click the **three dots (...)** in the upper right > **Board settings**.
6. Select **Columns** from the left navigation panel.
7. Click **Add column** to structure the board into 4 clear business validation stages:
 - Column 1: Ready for UAT Execution
 - Column 2: In Business Testing
 - Column 3: UAT Blocked (Defect Raised)
 - Column 4: Accepted by Business (Validated)
8. Map the corresponding issue statuses to each column and click **Back to board**.
9. Click **+ Create** in the top navigation bar to generate a test ticket:
 - *Issue Type:* UAT Test Case
 - *Summary:* UAT-03: Validate Multi-Entity Consolidated Balance Reporting
 - *Description:* Business User steps: Log in as Group CFO, aggregate accounts across 3 international subsidiaries, verify FX conversion parity, and export report.
10. Click **Create**.
11. Drag the ticket from Ready for UAT Execution to In Business Testing.
12. Simulate a business test failure: drag the ticket to UAT Blocked (Defect Raised).
13. Open the card, add a comment: UAT Execution Blocker: Consolidated report fails to sum subsidiary EUR accounts into USD total balance. Raising linked Sev 2 bug.
14. Observe how the dedicated board provides executive stakeholders with real-time visibility into business acceptance progress without cluttering the technical engineering sprint board.

3: Tracking Bugs, Severities, and Triage Processes

Activity 3.1: Designing a Multi-Tier Defect Severity & Priority Triage Matrix in Confluence

- **Tool:** Confluence
- **Objective:** Author an institutional Defect Management Standard defining exact technical severity categories, business priority SLAs, and a structured triage decision tree to eliminate ambiguity during incident escalations.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create** (+).
2. Title the page: [Your Name] - Defect Severity, Priority & Triage Standard (CMMI PPQA).
3. Use /heading 2 to add: 1. Defect Severity vs. Priority Governance Matrix.
4. Type /table and insert a 5-column, 5-row classification table:
 - Severity Level | Technical Impact Definition | Business Priority | Target Triage Window | Target Resolution SLA
 - Severity 1 (Blocker) | Complete production crash, data corruption, security breach, or total blockage of critical business path with no workaround. | P1 - Critical | < 15 Minutes | < 4 Hours (Emergency Hotfix)
 - Severity 2 (Critical) | Major functional failure affecting high-volume business capabilities, but an interim manual workaround exists. | P2 - High | < 1 Hour | < 24 Hours
 - Severity 3 (Major) | System functions normally, but non-critical features or secondary integrations fail intermittently. | P3 - Medium | < 4 Hours | Next Scheduled Sprint Release
 - Severity 4 (Minor / Cosmetic) | UI alignment flaws, typos, formatting quirks, or trivial usability discrepancies. | P4 - Low | Weekly Triage | Product Backlog Backlog Item
5. Below the table, use /heading 2 to add: 2. Daily Defect Triage Meeting Protocol.
6. Insert an ordered sequence detailing the 4-step triage agenda:
 - Step 1: Defect Verification - QA Lead validates reproducibility, system logs, and environment configurations.
 - Step 2: Severity & Priority Assignment - Tech Lead evaluates architectural severity; Product Owner establishes business priority.
 - Step 3: Action Determination - Committee assigns defect to: (A) Immediate In-Sprint Fix, (B) Deferred Backlog, or (C) Rejected (Working as Designed).
 - Step 4: CMMI CAR Root-Cause Tagging - PM tags defect with systemic source (e.g., Missing Requirement, Environment Drift, Unhandled Boundary).
7. Below the agenda, insert a **Warning Panel** (/panel) titled: Anti-Triage Rule:
 - Text: Severity ratings cannot be downgraded solely to meet release milestone dates. Any severity reclassification requires joint written approval from the QA Lead and Lead Systems Architect.
8. Click **Publish** in the top-right corner.

Activity 3.2: Configuring a CMMI-Compliant Defect Tracking & Triage Pipeline in Jira

- **Tool:** Jira Software
- **Objective:** Configure a Jira Bug issue type with custom root-cause categorization fields, severity ratings, and automated triage routing to fulfill CMMI *Measurement and Analysis (MA)* and *Causal Analysis and Resolution (CAR)* standards.

Step-by-Step Instructions:

1. In Jira Software, navigate to **Project settings > Issue types**.
2. Select the **Bug** issue type (or click **Add issue type > Bug**).
3. Click **Project settings > Automation** from the left navigation panel.
4. Click **Create rule** in the top-right corner to automate triage routing.
5. In the trigger search bar, type Issue created and select it. Click **Save**.
6. Click **+ New condition > Issue fields condition**:
 - *Field:* Issue Type
 - *Condition:* equals
 - *Value:* Bug
 - Click **Save**.
7. Click **+ New condition > Issue fields condition**:
 - *Field:* Priority (or custom field Severity)
 - *Condition:* equals
 - *Value:* Highest (or Sev 1 - Blocker)
 - Click **Save**.
8. Click **+ New action > Add flag** (flags the card red on all boards). Click **Save**.
9. Click **+ New action > Add comment**:
 - Enter the automated alert:

EMERGENCY TRIAGE ALERT: Severity 1 Defect logged.

- SLA: Triage response required within 15 minutes.

- Modern PM and Tech Lead notified for immediate war-room assessment.

- Click **Save**.
10. Name the rule: [Your Name] - Sev 1 Defect Triage & Escalation Rule.
 11. Click **Turn on rule**.
 12. Test the triage mechanism:
 - Click **+ Create** in the top navigation bar.
 - *Issue Type:* Bug
 - *Summary:* Database Deadlock on Concurrent Checkout Ledger Entries
 - *Priority:* Highest
 - *Description:* System logs throw SQL State 40P01 during peak concurrent orders. Repro rate: 100% under 50 threads.
 - Click **Create**.
 13. Open the newly created Bug ticket. Verify that the card is flagged, displays the automated triage alert comment, and contains links to capture systemic root causes for CMMI CAR reporting.

4: Establishing Clear Acceptance Criteria and Definition of Done (DoD)

Activity 4.1: Authoring an Organizational Definition of Done (DoD) Standard in Confluence

- **Tool:** Confluence
- **Objective:** Author an enterprise-wide, non-negotiable Definition of Done (DoD) standard that establishes objective verification criteria across code quality, automated testing, security, and documentation to fulfill CMMI *Process and Product Quality Assurance (PPQA)* audits.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create** (+).
2. Title the page: [Your Name] - Organizational Definition of Done (DoD) Standard (PAL-STD-005).
3. Use /heading 2 to add: 1. Purpose & Authority.
4. Add text: This document defines the mandatory, organization-wide Definition of Done (DoD). No user story, technical task, or defect fix can be moved to "Done" or included in a release candidate without meeting 100% of these objective verification baselines.
5. Use /heading 2 to add: 2. The Non-Negotiable DoD Quality Checklist.
6. Type /table and insert a 4-column, 6-row governance verification table:
 - Domain | Verification Requirement | Required Objective Evidence | CMMI Practice Area
 - Code Quality | Peer review approved by at least 1 Senior Engineer; zero unresolved review comments. | GitHub / Bitbucket Pull Request approval log | Verification (VER)
 - Automated Unit Testing | $\geq 80\%$ automated unit test branch coverage across newly created or modified code. | JaCoCo / Jest code coverage report attached | Verification (VER)
 - Static Security Analysis | SonarQube gate passed with 0 Blocker, 0 Critical bugs, and 0 Security Vulnerabilities. | SonarQube quality gate green badge | Technical Solution (TS)
 - Regression Verification | Automated CI regression suite passed cleanly with 0 failing test cases in Staging. | CI/CD pipeline build manifest | Product Integration (PI)
 - Documentation & Runbooks | API documentation updated via OpenAPI/Swagger; user-facing release notes drafted. | Confluence release notes page link | Configuration Management (CM)
7. Below the table, use /heading 2 to add: 3. DoD Tailoring & Governance Audit Protocol.
8. Insert an **Info Panel** (/panel) titled: PPQA Audit Enforcement:
 - Text: During sprint closure, the Modern Project Manager conducts a spot audit of 20% of closed user stories. Any story transitioned to "Done" without meeting all DoD criteria is reopened, flagged as process non-compliance in the CMMI PPQA audit log, and scheduled for immediate remediation.
9. Click **Publish** in the top-right corner.

Activity 4.2: Implementing Story-Level Acceptance Criteria Validation Gates in Jira

- **Tool:** Jira Software
- **Objective:** Implement a mandatory acceptance criteria sign-off checklist on Jira user stories using custom checklist items and workflow resolution controls, preventing stories from being marked "Done" prematurely.

Step-by-Step Instructions:

1. Open Jira Software and navigate to your project board.
2. In the top-right corner, click **+ Create** to generate a new user story:
 - *Issue Type:* Story
 - *Summary:* Implement Customer Address Verification Service via Google Maps API
 - *Story Points:* 5
3. In the **Description** field, enter formal, testable acceptance criteria using Gherkin syntax:

Acceptance Criteria Checklist:

- [] AC-1: Given a customer enters a valid postal address, when validation executes, then auto-complete returns verified zip and street coordinates.
- [] AC-2: Given an invalid or unrecognized address, when submitted, then the system highlights the address field in red and displays "Address not recognized".
- [] AC-3: Given the Google Maps API times out (>2000ms), when address lookup is invoked, then the system falls back gracefully to manual unverified entry without crashing.
- [] DoD Check: Unit test coverage exceeds 80%.
- [] DoD Check: Peer code review completed and approved.

4. Click **Create**.
5. Open the newly created story from your backlog or board.
6. Simulate incomplete development: leave checkboxes AC-1 and AC-2 checked, but leave **AC-3 (timeout fallback)** and the **DoD checks** unchecked.
7. Attempt to drag the story ticket across the board from In Testing to Done.
8. In the issue view, add a blocker comment: GATE AUDIT FAILED: Story cannot be marked Done. Acceptance Criterion AC-3 (Google Maps API timeout fallback) failed in staging. Returning ticket to In Development.
9. Move the ticket back to In Development to prevent premature increment closure, proving compliance with CMMI Level 3 Verification and DoD governance standards.

Metrics, Reporting, and Dashboards: Individual Hands-on Lab Guide

1: Configuring Actionable Project Dashboards in Jira

Activity 1.1: Building an Executive & Governance One-Pane-of-Glass Dashboard in Jira

- **Tool:** Jira Software
- **Objective:** Construct a multi-gadget executive telemetry dashboard in Jira using custom JQL filters to track active sprint health, high-severity defect density, and process compliance to satisfy CMMI *Measurement and Analysis (MA)* and *Project Monitoring and Control (PMC)* standards.

Step-by-Step Instructions:

1. Open Jira Software and click **Dashboards** in the top navigation bar.
2. Select **Create dashboard** from the dropdown menu.
3. Configure the dashboard properties:
 - **Name:** [Your Name] - Executive Delivery & CMMI Governance Hub
 - **Description:** Single-pane-of-glass tracking sprint commitments, critical defect leakage, and process compliance.
 - **Viewers:** Select Project: [Your Project] (or Private for sandbox evaluation).
4. Click **Save**.
5. Click **Change layout** in the top-right corner of the blank dashboard and select the **Two columns (split 50/50)** layout.
6. Click **+ Add gadget** in the top-right corner to open the gadget directory.
7. Locate and configure **Gadget 1 (Sprint Health)**:
 - Search for Sprint Health Gadget and click **Add**.
 - In the gadget settings, select your active Scrum board from the dropdown.
 - Check all visual telemetry indicators: *Time Elapsed*, *Work Complete*, *Scope Change*, and *Blockers*.
 - Set **Auto-refresh** to Every 15 minutes and click **Save**.
8. Locate and configure **Gadget 2 (Critical Defect Density)**:
 - Search for Filter Results and click **Add**.
 - Click the link inside the gadget to **Set up a new filter**, enter JQL: project = "[Your Project Key]" AND issuetype = Bug AND priority in (Highest, High) AND status != Done, and save the filter as Open Critical Defects.
 - Return to the dashboard gadget, select Open Critical Defects as the saved filter, set **Number of Results** to 5, and check columns: Key, Summary, Priority, Status, Assignee. Click **Save**.
9. In the right-hand column, locate and configure **Gadget 3 (Two-Dimensional Work-in-Progress Distribution)**:
 - Search for Two-Dimensional Filter Statistics and click **Add**.
 - Select a filter containing all active sprint issues (JQL: project = "[Your Project Key]" AND sprint in openSprints()).
 - Set **X-axis** to Assignee and **Y-axis** to Status.
 - Set **Show Totals** to Yes and click **Save**.

10. Below Gadget 3, locate and configure **Gadget 4 (Process Compliance / Missing Verification)**:
 - Search for Issue Statistics and click **Add**.
 - Select a saved filter for non-compliant stories (JQL: project = "[Your Project Key]" AND issuetype = Story AND status = "In Progress" AND (description !~ "Given" OR description !~ "Then")).
 - Set **Statistic Type** to Assignee and click **Save**.
11. Review the dashboard: verify that in less than 5 seconds you can identify overall sprint completion percentage, active blockers, unassigned critical bugs, and stories violating the Definition of Ready.

Activity 1.2: Automating Quantitative SLA & Blocker Metric Alerts in Jira

- **Tool:** Jira Software
- **Objective:** Configure an automated Jira subscription filter and escalation rule that calculates SLA breaches and alerts project leadership to work stagnation, fulfilling CMMI Level 4 *Quantitative Project Management (QPM)* monitoring practices.

Step-by-Step Instructions:

1. In Jira Software, click **Filters** in the top navigation bar and select **Advanced issue search**.
2. Switch to the **JQL** editor mode.
3. Enter the following quantitative threshold query to detect stagnant work items:

project = "[Your Project Key]" AND status in ("In Development", "In Review", "In Testing") AND
 updatedAt <= -48h

4. Click **Search** to run the query.
5. Click **Save as** above the search bar, name the filter: [Your Name] - 48h Work Stagnation SLA Alert, and click **Submit**.
6. Click **Details** next to the saved filter name, then click **New subscription**.
7. Set up the automated notification schedule:
 - *Recipients:* Select yourself (or Project Lead).
 - *Schedule:* Daily at 08:30 AM (prior to morning stand-up).
 - *Condition:* Check Subscribe even if no issues are found = **Unchecked** (only fires when an SLA breach occurs).
8. Click **Subscribe**.
9. Now configure a live workflow trigger: click **Project settings > Automation > Create rule**.
10. Set the **Trigger** to Scheduled:
 - Set the schedule to run every **1 day**.
 - Check **Run a JQL search and execute actions for each issue in the query**.
 - Paste the saved JQL query from Step 3 into the JQL box. Click **Save**.
11. Click **+ New action > Add flag** to place an impediment marker on the card.
12. Click **+ New action > Add comment**:

QUANTITATIVE GOVERNANCE SLA BREACH:

This work item has experienced zero progress or status updates for >48 hours during an active sprint. Under CMMI PMC/QPM guidelines, this issue is flagged for immediate triage during the morning stand-up.

13. Click **Save**, name the rule: [Your Name] - Stagnation SLA Auto-Flag, and click **Turn on rule**.

2: Analyzing Burndown, Burnup, and Cumulative Flow Diagrams

Activity 2.1: Diagnosing Delivery Anti-Patterns across Sprint Burndown & CFD in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Analyze visual curves from sample Sprint Burndown, Release Burnup, and Cumulative Flow Diagrams (CFD) to identify delivery anti-patterns (such as late scope injection, QA bottlenecks, and mini-waterfalls) and design corrective governance interventions.

Step-by-Step Instructions:

1. Open Miro, click **+ New board** > **Create a blank board**, and title it: [Your Name] - Agile Telemetry & Anti-Pattern Diagnostic Lab.
2. Add a prominent header across the top using the **Text** tool (T): Agile Metric Diagnostic & Root Cause Matrix (CMMI MA / CAR).
3. Create 3 large rectangular sections side by side using the **Shape** tool (R):
 - Section 1: Sprint Burndown Diagnostic (Commitment Variance)
 - Section 2: Release Burnup Diagnostic (Scope Creep Analysis)
 - Section 3: Cumulative Flow Diagram (CFD Bottleneck Analysis)
4. In **Section 1 (Burndown)**, draw a sample coordinate chart:
 - Draw an ideal linear dashed line sloping downward from 40 points on Day 1 to 0 points on Day 10.
 - Draw an actual line that stays flat at 40 points from Day 1 to Day 8, and suddenly drops vertically to 10 points on Day 10 ("The Cliff Drop").
 - Attach a red sticky note (N): Anti-Pattern: "Mini-Waterfall" inside the sprint. Developers hoarded tickets without testing until the final 48 hours; QA was crushed; 10 points missed.
 - Attach a green sticky note: Corrective Action: Implement WIP limits in Jira and enforce daily incremental pull request merges.
5. In **Section 2 (Burnup)**, draw two lines:
 - Line A (Top - Red): Total Scope Line — starts at 100 points, steps upward to 120 points in Sprint 2, and steps to 150 points in Sprint 3.
 - Line B (Bottom - Blue): Completed Work Line — steady 30-point velocity per sprint.
 - Attach a red sticky note: Anti-Pattern: Runaway Scope Creep. Delivery velocity is healthy, but the release date slips because stakeholders inject unbudgeted scope.
 - Attach a green sticky note: Corrective Action: Lock Fix Version baseline; enforce the CMMI Scope-Swap rule (new story in requires equivalent points out).
6. In **Section 3 (CFD)**, draw a cumulative area chart showing workflow bands: Done, In QA, In Dev, Backlog:
 - Illustrate the band for In QA widening dramatically across the middle of the project timeline while the Done line flattens.
 - Attach a red sticky note: Anti-Pattern: QA Capacity Bottleneck. Lead time is exploding because engineering throughput exceeds verification bandwidth.
 - Attach a green sticky note: Corrective Action: Apply Little's Law (Lead Time = WIP / Throughput); impose a strict QA column WIP limit of 4 tickets; swarm developers on automated testing.
7. Below the 3 sections, create an executive summary banner: CMMI Level 4 Takeaway: Visual flow metrics diagnose systemic organizational dysfunctions that simple velocity numbers conceal.
8. Export the board as an image or generate a shareable view link for submission.

Activity 2.2: Generating, Interpreting, and Auditing Agile Reports in Jira

- **Tool:** Jira Software
- **Objective:** Generate live Burndown, Burnup, and Cumulative Flow Diagrams in Jira, calculate quantitative flow metrics (Lead Time, Cycle Time, and Commitment Reliability), and author a CMMI *Measurement and Analysis (MA)* variance report.

Step-by-Step Instructions:

1. In Jira Software, navigate to your active Scrum project.
2. In the left-hand navigation sidebar, click on **Reports**.
3. Select **Burndown Chart** from the available reports:
 - Review the chart for your active/recent sprint.
 - Locate the **Issue event table** below the graph.
 - Identify any entries highlighted with a red upward line or listed as Issue added to sprint after the sprint start date.
 - Record the net scope change percentage:
$$\text{Scope Change \%} = \left(\frac{\text{Points Added Mid-Sprint}}{\text{Initial Committed Points}} \right) \times 100$$
4. Return to the **Reports** menu and click **Cumulative Flow Diagram**:
 - In the top-right filter options, set the timeframe to **Past 3 months** (or current sprint duration).
 - Refine the columns: ensure To Do, In Progress, In Review, Testing, and Done are selected.
 - Observe the band widths: determine which workflow state represents the widest vertical band (the primary process constraint).
5. Open your project space in **Confluence** and click **Create (+)**.
6. Title the page: [Your Name] - Quantitative Process Performance Audit (Sprint Telemetry).
7. Use /table to insert a 4-column, 5-row quantitative variance table:
 - Agile Metric | Target Plan / Baseline | Actual Observed Value | Variance / CMMI Action Trigger
 - Commitment Reliability | 100% of committed points | 82.5% (33 of 40 pts) | -17.5% (Violates 10% threshold; trigger CAR review)
 - Mid-Sprint Scope Injection | 0 Points (Frozen baseline) | 8 Points added | Scope creep breach; requires CCB trade-off audit
 - Average Cycle Time | < 3.0 Days per story | 5.2 Days | WIP limits exceeded; enforce column throttling
 - Escaped Defect Ratio | < 5% of delivered points | 2.5% (1 minor bug) | WITHIN CONTROL LIMITS (Level 4 Stable)
8. Below the table, insert an **Info Panel** (/panel) titled: CMMI MA Auditor Attestation:
 - Text: Telemetry data extracted directly from Jira reporting engine without manual modification. Historical control charts confirm that delivery variance is driven by mid-sprint scope injection rather than engineering throughput degradation.
9. Click **Publish** in the top-right corner.

Change Management and Scope Creep: Individual Hands-on Lab Guide

1: Implementing Formal Change Request and Approval Processes

Activity 1.1: Configuring an Auditable Change Control Board (CCB) Workflow in Jira

- **Tool:** Jira Software
- **Objective:** Configure a specialized Change Request issue type, screen, and workflow in Jira that enforces mandatory impact evaluations and formal CCB approval gates to satisfy CMMI *Configuration Management (CM)* and *Requirements Management (REQM)* standards.

Step-by-Step Instructions:

1. Open Jira Software and click **Project settings** (gear icon) in the bottom-left sidebar.
2. Select **Issue types** > click **Add issue type**.
3. In the modal, enter:
 - *Name:* Change Request
 - *Description:* Formal request to alter approved scope, architectural baseline, or milestone dates.
 - *Type:* Standard Issue Type
4. Click **Create**.
5. In the left navigation menu, select **Workflows** > click the **Pencil (Edit)** icon next to your active project workflow (or create a custom workflow for Change Request).
6. Build a 5-stage formal CCB lifecycle:
 - Status 1: Submitted
 - Status 2: Impact Analysis
 - Status 3: Pending CCB Approval
 - Status 4: Approved for Implementation
 - Status 5: Rejected / Shelved
7. Click the transition arrow connecting Pending CCB Approval to Approved for Implementation.
8. In the right-hand panel, select **Conditions** > click **Add condition**.
9. Select **Permission Condition** (or **User In Role Condition**), set the required role to Project Administrators (or CCB Committee), and click **Add**.
10. Click **Validators** on the same transition > click **Add validator** > select **Field Required Validator**.
11. Mark Resolution and Fix versions as mandatory fields upon approval. Set the error message: CMMI CM RULE: Approved changes must be mapped to an authorized release baseline.
12. Click **Publish Draft** in the top yellow workflow banner.
13. Return to your board, click **+ Create**, and log a test change:
 - *Issue Type:* Change Request
 - *Summary:* CR-104: Expand KYC Document Parser to Support Driver Licenses
 - *Description:* Commercial client demands adding US/EU driver licenses to passport-only baseline.
 - *Priority:* High

14. Open the created ticket, advance the status to Impact Analysis, and add an evaluation comment:
 - **IMPACT ASSESSMENT:** Requires 13 story points across OCR engine; extends Sprint 7 delivery by 4 calendar days. Database schema impacts: Low.
15. Transition the ticket to Pending CCB Approval, verify that approval controls require administrator sign-off, and move the ticket to Approved for Implementation.

Activity 1.2: Authoring a Centralized CCB Charter and Change Register in Confluence

- **Tool:** Confluence
- **Objective:** Establish an enterprise Change Control Board (CCB) operating standard and audit-ready Change Log register in Confluence to provide an unbroken paper trail of baseline changes for CMMI Level 3 appraisals.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create** (+).
2. In the page title field, enter: [Your Name] - Change Control Board (CCB) Charter & Baseline Register.
3. Use /heading 2 to add: 1. CCB Governance Framework & Operating Standard.
4. Type /table and insert a 3-column, 4-row governance summary table:
 - Governance Dimension | Policy Specification | CMMI Process Area
 - CCB Composition | Modern Project Manager (Chair), Product Owner, Lead Enterprise Architect, Business Sponsor. | Configuration Management (CM)
 - Review Cadence | Weekly scheduled review, or emergency fast-track review (<24h SLA) for Sev 1 blockers. | Project Monitoring & Control (PMC)
 - Approval Threshold | Unanimous consent required for baseline schedule, cost, or regulatory architectural shifts. | Requirements Management (REQM)
5. Use /heading 2 to add: 2. Auditable Change Log Register.
6. Type /table and insert an 8-column, 3-row change ledger:
 - Headers: CR ID | Date Submitted | Originator / Sponsor | Scope Description | Schedule Impact | Cost / Point Impact | CCB Decision | Audit Evidence Link
 - Row 1: CR-101 | Oct 12, 2026 | VP of Commercial Banking | Add biometric FaceID support to mobile login. | +0 Days (Neutral via Scope-Swap) | 8 Story Points | APPROVED | Jira: CR-101 (Approved)
 - Row 2: CR-102 | Oct 18, 2026 | Chief Marketing Officer | Redesign global navigation bar with custom animated UI. | +7 Days (Schedule Breach) | 13 Story Points | REJECTED (Shelved to Post-MVP) | CCB Minutes: Log-04
7. Below the table, type /panel and insert a **Warning Panel** titled: Baseline Control Rule:
 - Text: No developer or technical lead is authorized to pull unapproved Change Request items into an active sprint backlog. Work items implemented without CCB approval constitute a Level 3 process non-compliance violation.
8. Click **Publish** in the top-right corner.

2: Negotiating Scope Alterations Against Time and Budget Constraints

Activity 2.1: Building a Scope-Swap Trade-Off Calculator in Confluence

- **Tool:** Confluence
- **Objective:** Construct a quantitative scope-swap calculation model in Confluence that enforces the Iron Triangle (fixed time and fixed capacity), demonstrating how to accommodate critical new requests without slipping project release dates.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - Iron Triangle Scope-Swap Negotiation Model.
3. Use /heading 2 to add: 1. Negotiation Principle: Value-Neutral Scope Swapping.
4. Add text: When delivery deadlines and team capacity are fixed by commercial contract, scope is the sole variable parameter. To introduce new emergency features into an active release candidate without compromising delivery dates or inducing burnout, an identical volume of lower-priority scope must be removed from the release baseline.
5. Use /heading 2 to add: 2. Quantitative Scope-Swap Reconciler.
6. Type /table and insert a 4-column, 4-row incoming vs. outgoing scope table:
 - **Incoming Scope (Demanded by Business):**
 - Jira Ticket ID | Feature Description | MoSCoW Priority | Story Points Demanded
 - CR-104 | Driver License OCR Extraction Engine | Must-Have (New Mandate) | 13 Points
 - **Outgoing Compensatory Scope (Removed from Release Baseline):**
 - Jira Ticket ID | Feature Description | MoSCoW Priority | Story Points Deferred
 - STORY-88 | Export Transaction History to Excel / CSV | Could-Have | 5 Points
 - STORY-92 | Custom Customer Profile Avatar Upload | Could-Have | 3 Points
 - STORY-96 | Dark Mode Theme UI Support | Won't-Have (This Release) | 5 Points
7. Below the table, type /table and insert a 2-column reconciliation check:
 - Incoming Scope Total: | +13 Story Points
 - Outgoing Descope Total: | -13 Story Points
 - Net Capacity Impact: | **0 Story Points (Schedule Baseline Preserved)**
 - Impact on Release Date: | **0 Days Variance**
8. Below the calculation, insert a **Success Panel** (/panel) titled: Executive Trade-Off Alignment Agreement:
 - Text: The Business Sponsor accepts that by approving CR-104 (Driver License OCR), CSV Export, Profile Avatars, and Dark Mode are officially deferred to Release 2.0. The release date remains locked for November 15, 2026.
 - Add signature lines: Modern Project Manager: [Signed] | Business Sponsor: [Signed].
9. Click **Publish** in the top-right corner.

Activity 2.2: Reallocating Sprints and Version Baselines in Jira

- **Tool:** Jira Software
- **Objective:** Execute an approved scope-swap negotiation directly inside Jira by reprioritizing backlog items, reassigning Fix Versions, and verifying sprint capacity equilibrium.

Step-by-Step Instructions:

1. Open Jira Software and navigate to your project's **Backlog** view.
2. In the left panel, toggle the **Versions** panel to open (or use the **Versions** dropdown).
3. Confirm that Release 1.0 - MVP is selected and active. Note the total estimated story points currently assigned to this version.
4. Locate the newly approved change request ticket: CR-104: Expand KYC Document Parser to Support Driver Licenses (13 Story Points).
5. Open CR-104, set the **Fix version** to Release 1.0 - MVP, and drag it directly into the active/upcoming sprint backlog container.
6. Observe the sprint capacity indicator turn red or show an over-allocation warning (e.g., committed points increase from 32 to 45).
7. Execute the compensatory descope: locate the 3 agreed lower-priority user stories in the sprint backlog:
 - STORY-88: Export Transaction History to CSV (5 pts)
 - STORY-92: Custom Customer Profile Avatar Upload (3 pts)
 - STORY-96: Dark Mode Theme UI Support (5 pts)
8. Open each of these 3 tickets individually:
 - Remove Release 1.0 - MVP from the **Fix versions** field.
 - Assign them to Release 2.0 - Post-MVP Backlog.
 - Drag them out of the sprint container down into the general product backlog.
9. Review the sprint container header: confirm that total planned points return to the target commitment baseline (32 points).
10. Open STORY-88 and add a comment: DESCOPE AUDIT: Removed from Release 1.0 baseline and deferred to Release 2.0 per approved CCB Scope-Swap for CR-104. Click **Save**.

3: Maintaining Alignment with Core Business Objectives

Activity 3.1: Constructing a Strategic OKR-to-Backlog Alignment Map in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Design an alignment flowchart mapping strategic enterprise Objectives and Key Results (OKRs) directly to Epics and User Stories, creating a visual filter that eliminates non-aligned scope creep ("gold-plating").

Step-by-Step Instructions:

1. Open Lucidchart, click **+ New > Lucidchart > Blank Document**, and title it: [Your Name] - Strategic OKR Alignment & Anti-Gold-Plating Blueprint.
2. From the shape menu, drag a **Horizontal Swimlane** container onto the canvas with **3 swimlanes**:
 - Label Swimlane 1: Strategic Business Drivers (Enterprise OKRs)
 - Label Swimlane 2: Authorized Product Epics (Release Baselines)
 - Label Swimlane 3: Engineering Scope Triage (Gold-Plating Filter Gate)
3. Color-code the lanes: Swimlane 1 dark blue, Swimlane 2 light blue, and Swimlane 3 light yellow.
4. In **Swimlane 1 (OKRs)**, place 2 process blocks:
 - Block A: Strategic Objective: Expand Corporate Onboarding Throughput
 - Block B: Key Result (KR 1.2): Cut Corporate Customer Onboarding Time from 14 Days to <48 Hours
5. Connect Block A to Block B with a solid directional arrow.
6. In **Swimlane 2 (Epics)**, place an authorized epic block aligned directly beneath KR 1.2:
 - Block Label: Epic 4: Automated Identity & Corporate Document Verification Service
7. Connect KR 1.2 in Swimlane 1 to Epic 4 in Swimlane 2.
8. In **Swimlane 3 (Triage)**, place a decision diamond centered horizontally:
 - Diamond Label: Strategic Alignment Filter: Does Story Directly Move KR 1.2?
9. Below the diamond, place two outcome blocks:
 - Block 1 (Green Rectangle): Authorized Scope: User Story 4.1 - Instant Tax Identification Lookup API (Reduces time by 3 days) -> ACCEPT INTO SPRINT
 - Block 2 (Red Rectangle): Gold-Plating Scope: User Story 4.8 - 3D Animated Interactive Loading Spinner on ID Upload Screen -> REJECT / REMOVE
10. Draw connecting arrows from Epic 4 to the Decision Diamond, and from the Diamond to the outcome blocks:
 - Arrow to Block 1: Label YES (Measurable Metric Impact).
 - Arrow to Block 2: Label NO (Zero Strategic Impact / Gold-Plating).
11. Add a callout box attached to Block 2: CMMI DAR Principle: Engineering enhancements that do not advance authorized business objectives consume budget without ROI and must be eliminated.
12. Click **File > Export > PNG** (or copy the shareable link) for submission.

Activity 3.2: Facilitating a Strategic Scope-Rejection Simulation in MS Teams

- **Tool:** MS Teams
- **Objective:** Conduct a simulated stakeholder alignment review in MS Teams, applying objective business criteria to professionally reject an executive's out-of-scope feature request while protecting team focus and CMMI process baselines.

Step-by-Step Instructions:

1. Open MS Teams, navigate to your project team, and click the three dots (...) > **Add channel**.
2. Set the Channel name to: governance-steering-committee.
3. Set the privacy to **Standard** and click **Create**.
4. In the channel posts area, click **New post**, expand the formatting editor, and select **Announcement**.
5. Set the Headline banner to: SCOPE ALIGNMENT TRIAGE: REQUEST FOR AI CHATBOT IN RELEASE 1.0.
6. Simulate an executive request by pasting this message:

****EXECUTIVE SCOPE REQUEST (From Head of Digital Innovation):****

"Team, we need to embed a generative AI conversational chatbot directly into the account dashboard for the Release 1.0 launch next month. It will make the product look cutting-edge for our upcoming investor conference. Please pull this into the current sprint immediately."

7. Click **Post**.
8. Click **Reply** directly under the announcement to author the **Project Manager / IT Director Strategic Response**:

****MODERN PROJECT MANAGER - STRATEGIC ALIGNMENT DETERMINATION:****

Thank you for the suggestion. Before altering our active production baseline, we evaluated this request against our CMMI Requirements Management (REQM) standard and contractual launch baselines:

1. ****Strategic Metric Alignment:****

- Release 1.0 Core Driver: *Achieve compliant, secure multi-currency transaction execution (KR 1.1).*
- Evaluation: A generative chatbot does not advance transaction clearance SLAs or regulatory compliance.

2. ****Delivery & Risk Impact:****

- Effort: Estimated at 21 Story Points.
- Impact: Pulling this into the active release would require descoping the Core Settlement Audit Logger or slipping the hard launch date by 3 weeks.
- Architectural Risk: Introduces unvetted third-party LLM data leakage risks prior to InfoSec audit.

3. ****Binding Recommendation & Next Steps:****

- ****Action:**** REJECT for Release 1.0 baseline.
- ****Alternative:**** Logged as Epic 12 in the Discovery Backlog. We will run an initial feasibility spike during Release 2.0 discovery once core transaction functionality is in production.

9. Click the paper airplane icon to post the response.
10. Click the three dots (...) on your response, select **Pin**, and confirm the pin.
11. Verify that the thread demonstrates a defensible, objective, and professional scope negotiation that maintains stakeholder respect while safeguarding baseline integrity.

CMMI in Delivery, Quality, and Governance: Individual Hands-on Lab Guide

1: Using CMMI KPAs to Strengthen QA, UAT, and Release Management

Activity 1.1: Building an End-to-End CMMI Practice Area Traceability Matrix in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Architect a visual cross-reference pipeline mapping CMMI Practice Areas (*Verification, Validation, Configuration Management, and Process & Product Quality Assurance*) directly into daily QA, UAT, and Release management checkpoints.

Step-by-Step Instructions:

1. Open Lucidchart, click **+ New > Lucidchart > Blank Document**, and title it: [Your Name] - CMMI KPA-to-Delivery Pipeline Architecture.
2. Drag a **Horizontal Swimlane** container onto the canvas and set it to **4 swimlanes**:
 - Label Swimlane 1: In-Sprint QA & Static Testing (Verification - VER)
 - Label Swimlane 2: Business User Testing & Staging (Validation - VAL)
 - Label Swimlane 3: Release Packaging & Baseline Control (Configuration Management - CM)
 - Label Swimlane 4: Quality Governance & Process Audits (PPQA)
3. In **Swimlane 1 (VER)**, drag and place 2 process blocks:
 - Block A: Static Code Analysis & Unit Test Execution Gate (>80% coverage)
 - Block B: Peer Code Review & Pull Request Inspection (>=1 Senior Approver)
4. In **Swimlane 2 (VAL)**, place 2 process blocks:
 - Block A: Business Acceptance Test Scenario Execution in Staging Sandbox
 - Block B: Operational Acceptance Sign-Off by Product Owner & Business SME
5. In **Swimlane 3 (CM)**, place 2 process blocks:
 - Block A: Cryptographic SHA-256 Hashing & Immutable Container Tagging (SemVer)
 - Block B: Production Deployment Manifest & Automated Environment Audit
6. In **Swimlane 4 (PPQA)**, place 3 diamond shapes (Audit Checkpoints) positioned vertically beneath Swimlanes 1, 2, and 3:
 - Diamond 1: PPQA Gate 1: Story DoD Compliance Audit
 - Diamond 2: PPQA Gate 2: UAT Exit Criteria & Defect Zero-Tolerance Audit
 - Diamond 3: PPQA Gate 3: Release Manifest Baseline Audit
7. Draw vertical dashed connector lines from the process blocks in Swimlanes 1–3 down to their respective PPQA audit diamonds in Swimlane 4.
8. Color the process blocks light blue and the PPQA audit diamonds light purple.

9. Click **File > Export > PNG** (or copy the share link) for submission.

Activity 1.2: Automating CMMI PPQA Quality Audit Workflows in Jira

- **Tool:** Jira Software
- **Objective:** Configure an automated Jira rule that enforces Process & Product Quality Assurance (PPQA) by automatically verifying that user stories have linked verification test cases before transitioning to "Ready for Release."

Step-by-Step Instructions:

1. Open Jira Software and navigate to your project.
2. Click **Project settings** (gear icon) in the bottom-left sidebar > **Automation**.
3. Click **Create rule** in the upper-right corner.
4. Set the **Trigger**:
 - Search for Issue transitioned and select it.
 - Set **From status** to In Testing (or In Review).
 - Set **To status** to Ready for Release (or Done).
 - Click **Save**.
5. Click **+ New condition > Issue fields condition**:
 - *Field*: Issue Type
 - *Condition*: equals
 - *Value*: Story
 - Click **Save**.
6. Click **+ New condition > Related issues condition**:
 - *Related issues*: Linked issues
 - *Link type*: is tested by (or tests / is tested by)
 - *Condition*: None match JQL
 - *JQL*: issuetype in ("UAT Test Case", Test, Bug) AND status = Done
 - Click **Save**.
7. Click **+ New action > Transition issue**:
 - Set target status back to In Testing.
 - Click **Save**.
8. Click **+ New action > Add comment**:
 - Enter the governance message:

CMMI PPQA NON-COMPLIANCE AUDIT:

This story cannot transition to 'Ready for Release'. Under CMMI Verification (VER) and PPQA standards, every functional story must have at least one verified test execution ticket linked and marked 'Done'.

- Click **Save**.
9. Name the rule: [Your Name] - PPQA Automated Verification Enforcer.
 10. Click **Turn on rule**.
 11. Test the automation: navigate to your board, create a story without linked test cases, attempt to move it to Ready for Release, and confirm that the rule reverses the transition and adds the PPQA audit comment.

2: Establishing Measurable KPIs Aligned with Maturity Goals

Activity 2.1: Designing a Maturity Tier KPI Framework in Confluence

- **Tool:** Confluence
- **Objective:** Author an institutional Process Performance Metric Standard that defines specific quantitative KPIs aligned with CMMI Maturity Levels 2 through 5, outlining collection methods and management thresholds.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - CMMI Maturity Tier KPI Governance Standard (PAL-STD-008).
3. Use /heading 2 to add: 1. Organizational Metric Framework.
4. Add text: This standard defines the mandatory Key Performance Indicators (KPIs) required to demonstrate quantitative control and progressive process maturity under CMMI appraisals.
5. Type /table and insert a 5-column, 5-row governance matrix:
 - CMMI Maturity Tier | Target Metric / KPI | Calculation Formula | Data Collection Source | Target Control Threshold
 - Level 2: Managed | Commitment Reliability Ratio (CRR) | (Delivered Points / Committed Points) * 100 | Jira Sprint Burndown | $\geq 85\%$ to $\leq 110\%$
 - Level 3: Defined | Defect Removal Efficiency (DRE) | (Bugs found pre-release / Total bugs found) * 100 | Jira Defect Log | $\geq 90\%$ removed pre-release
 - Level 4: Quantitatively Managed | Defect Injection Rate (DIR) | Number of Escaped Bugs / Delivered Story Points | Jira Release Telemetry | ≤ 0.05 defects / point
 - Level 5: Optimizing | Causal Action Implementation Velocity | Days from Incident Post-Mortem to CI/CD Guardrail Push | Confluence CAR Log + Git | ≤ 5 Business Days
6. Below the table, use /heading 2 to add: 2. Level 4 Statistical Control Limits.
7. Type /panel and insert an **Info Panel** detailing process limits:
 - Process Performance Model (PPM) Rules:
 - Any sprint where CRR deviates outside the 85%-110% control band triggers a formal Project Monitoring and Control (PMC) variance report.
 - Any release where Defect Injection Rate exceeds 0.08 triggers an immediate root-cause freeze under Causal Analysis and Resolution (CAR).
8. Click **Publish** in the top-right corner.

Activity 2.2: Building a CMMI Quantitative Process Dashboard in Jira

- **Tool:** Jira Software
- **Objective:** Construct a dedicated Jira dashboard tracking quantitative process metrics across velocity variance, defect removal efficiency, and pull-request verification compliance.

Step-by-Step Instructions:

1. In Jira Software, click **Dashboards > Create dashboard**.
2. Set the Name to: [Your Name] - CMMI Process Performance Telemetry (Level 3-4).
3. Set Viewers to your active project or private sandbox, and click **Save**.
4. Set the layout to **Two columns**.
5. Add **Gadget 1 (Level 2 Schedule & Commitment Stability)**:
 - Search for Sprint Burndown Gadget > click **Add**.

- Configure it to display your current active sprint board. Click **Save**.
- 6. Add **Gadget 2 (Level 3 Defect Containment)**:
 - Search for Created vs. Resolved Chart > click **Add**.
 - Set Project to your active project.
 - Set Issue Types to Bug.
 - Set Period to Weekly for the past 90 days.
 - Set Cumulative to Yes and click **Save**. (Demonstrates whether defect resolution outpaces defect creation).
- 7. Add **Gadget 3 (Level 3 Verification Peer Review Compliance)**:
 - Search for Filter Results > click **Add**.
 - Set saved filter to: project = "[Your Project Key]" AND issuetype = Story AND status = Done AND labels != "Peer-Reviewed".
 - Name of filter: Stories Missing Mandatory Peer Review.
 - Display columns: Key, Summary, Assignee, Updated. Click **Save**.
- 8. Add **Gadget 4 (Level 4 Work Item Defect Distribution)**:
 - Search for Pie Chart > click **Add**.
 - Select your project, and set **Statistic Type** to Priority.
 - Click **Save**.
- 9. Verify that the dashboard offers real-time quantitative evidence of process control without manual spreadsheet calculation.

3: Ensuring Consistent Acceptance Criteria and Definition of Done (DoD)

Activity 3.1: Constructing a Multi-Tier Definition of Done Standardization Grid in Confluence

- **Tool:** Confluence
- **Objective:** Author an enterprise standard Definition of Done (DoD) in Confluence that defines non-negotiable organizational baselines and establishes clear governance rules for team-level tailoring.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - Enterprise DoD Framework & Team Tailoring Guide.
3. Use /heading 2 to add: 1. The Non-Negotiable Core Baseline (Organizational Standard).
4. Type /table and insert a 4-column, 5-row table:
 - Quality Pillar | Mandatory Criteria | Verification Method | CMMI Area
 - Code Quality | Peer review approved with zero unresolved comments. | Bitbucket / GitHub PR Approval | VER
 - Test Coverage | >= 80% automated unit and component branch coverage. | SonarQube / JaCoCo Report | VER
 - Security Scans | 0 Critical, 0 High static vulnerabilities; dependencies updated. | Snyk / OWASP Scan | TS
 - Documentation | API endpoints documented in OpenAPI; release notes updated. | Confluence Release Log | CM
5. Use /heading 2 to add: 2. Team-Specific Permitted Tailoring Boundaries.

6. Insert a 3-column table showing how specialized delivery teams extend (but never dilute) the baseline:
 - Delivery Stream | Permitted Additional DoD Criteria | Tailoring Justification
 - Mobile Applications Team | Must pass automated device farm testing across iOS and Android top 10 devices. | Device fragmentation risks.
 - Core Data & Settlement Team | Must include automated rollback migration script tested against production data replica. | High data loss financial risk.
 - Microservices API Team | Must include updated contract tests using Pact framework. | Cross-service dependency coupling.
7. Below the tables, insert a **Warning Panel** (/panel):
 - Heading: Governance Tailoring Constraint:
 - Text: No team has the authority to waive or reduce the 80% test coverage or peer review requirements. Any dilution of the baseline constitutes a CMMI PPQA non-conformance finding.
8. Click **Publish** in the top-right corner.

Activity 3.2: Enforcing Acceptance Criteria Checklists in Jira via Field Configurations

- **Tool:** Jira Software
- **Objective:** Configure custom acceptance criteria checklist fields and issue templates in Jira, ensuring user stories cannot be saved or closed without complete Gherkin validation scenarios.

Step-by-Step Instructions:

1. In Jira Software, navigate to **Project settings > Issue types**.
2. Select **Story**.
3. Locate the **Description** field configuration (or add a custom multiline field named Acceptance Criteria).
4. Click **Project settings > Automation > Create rule**.
5. Set the **Trigger**:
 - Search for Issue created and select it. Click **Save**.
6. Click **+ New condition > Issue fields condition**:
 - *Field:* Issue Type
 - *Condition:* equals
 - *Value:* Story
 - Click **Save**.
7. Click **+ New condition > Issue fields condition**:
 - *Field:* Description
 - *Condition:* does not contain
 - *Value:* Given
 - Click **Save**.
8. Click **+ New action > Add comment**:

QUALITY GATE VIOLATION:

This story does not meet the organizational Definition of Ready (DoR).

Every user story must contain formal acceptance criteria utilizing 'Given-When-Then' structure before it can be scheduled for sprint planning.

- Click **Save**.

9. Click + **New action** > **Edit issue**:
 - Set field Labels > Add > Missing-Acceptance-Criteria.
 - Click **Save**.
10. Name the rule: [Your Name] - Enforce Story Acceptance Criteria Structure.
11. Click **Turn on rule**.
12. Verify by creating a test story with plain text (e.g., Update user profile page). Observe that Jira automatically flags the issue and applies the non-compliance label.

4: How CMMI Supports Continuous Improvement and Post-Release Learning Cycles

Activity 4.1: Building a Closed-Loop CMMI CAR Root-Cause Workflow in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Design an interactive CMMI Causal Analysis and Resolution (CAR) process blueprint in Miro that tracks an escaped production defect from post-mortem investigation to permanent organizational process asset updates.

Step-by-Step Instructions:

1. Open Miro, click + **New board** > **Create a blank board**, and title it: [Your Name] - CMMI CAR Closed-Loop Learning System.
2. Draw a large circular or horizontal 5-stage continuous improvement cycle using the **Shape** tool (R):
 - Stage 1: Defect Detection & Incident Containment
 - Stage 2: Blameless 5-Whys Root Cause Analysis
 - Stage 3: Systemic Process Deficiency Identification
 - Stage 4: Automated Preventive Action Engineering
 - Stage 5: Process Asset Library (PAL) & CI/CD Gate Institutionalization
3. Connect the 5 stages sequentially using thick directional arrows.
4. Below Stage 2, attach a yellow sticky note (N) detailing the case study:
 - Incident: Production batch payment failure due to unhandled special characters in user names.
 - Why 1: XML parser failed on ampersand character.
 - Why 2: Integration tests only used standard alphanumeric mock data.
 - Why 3: No automated fuzz-testing suite exists in the CI pipeline.
5. Below Stage 4, attach a green sticky note:
 - Engineering Action: Implement OWASP automated fuzz-testing tool into Jenkins/GitLab pipeline.
6. Below Stage 5, attach a blue sticky note:
 - Institutional CAR Update: Updated PAL-STD-005 (Definition of Done) to mandate that all new API endpoints must pass automated fuzz-testing with zero crashes before production merge.
7. Draw a feedback connector line looping from Stage 5 all the way back to Stage 1 labeled: Prevents Defect Class Across All Future Projects.
8. Export the board as an image or generate a share link for submission.

Activity 4.2: Logging Institutional CAR Preventive Actions in Jira and Confluence

- **Tool:** Jira Software & Confluence
- **Objective:** Capture a production post-incident review in Confluence and automate the generation of trackable engineering remediation backlog tickets in Jira to fulfill CMMI *Causal Analysis and Resolution (CAR)* audit standards.

Step-by-Step Instructions:

1. In Confluence, open your space, click **Create (+)**, and title the page: [Your Name] - CAR Dossier: Post-Incident Review #108.
2. Insert a 2-column table summarizing the incident:
 - Incident Key: | INC-108: Payment Gateway Timeout Outage
 - Root Cause Class: | Deficient Boundary Condition in Timeout Circuit Breaker
 - CMMI Practice Area: | Causal Analysis and Resolution (CAR SP 1.2)
3. Use `/jira` on the Confluence page to select **Create new issue:**
 - *Project:* Your assigned project.
 - *Issue Type:* Task
 - *Summary:* [CAR-ACT-108] Embed Automated Fuzz-Testing Gate into Core Payment Pipeline
 - *Description:* Permanent process remediation resulting from INC-108 post-mortem. Pipeline must fail any PR that crashes under special character payloads.
4. Click **Insert**.
5. Switch to Jira Software and open the newly created task [CAR-ACT-108].
6. Add the label CMMI-CAR-Action and assign it a priority of High.
7. Link this task to your upcoming sprint backlog container.
8. Add a comment: CMMI CAR AUDIT RECORD: This engineering task represents a permanent preventive process update resulting from PIR #108. Completion is tracked as part of organizational maturity improvements.
9. Verify that the task is embedded bidirectionally between Confluence and Jira.

5: Preparing Teams for CMMI Appraisals (SCAMPI A/B/C)

Activity 5.1: Designing a SCAMPI Appraisal Hierarchy & Evidence Map in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Design an appraisal readiness diagram comparing SCAMPI Class A, B, and C appraisal scopes, and map the required objective evidence streams (Direct Work Products, Indirect Work Products, and Affirmations).

Step-by-Step Instructions:

1. Open Lucidchart, click **+ New > Lucidchart > Blank Document**, and title it: [Your Name] - SCAMPI Appraisal Architecture & Evidence Model.
2. In the top half of the canvas, draw 3 horizontal tiered containers:
 - Tier 1 (Top): Class C Appraisal (Informal Gap Analysis - Spot Checks & Interviews)
 - Tier 2 (Middle): Class B Appraisal (Pre-Appraisal / Dry Run - Process Compliance Testing)
 - Tier 3 (Bottom): Class A Appraisal (Formal Benchmark Appraisal - Full Rating Determination)
3. Color Tier 1 light yellow, Tier 2 light orange, and Tier 3 light red (indicating increasing rigor).

4. In the bottom half of the canvas, create 3 vertical rectangular boxes detailing **Mandatory SCAMPI Evidence Types**:
 - Box 1: Direct Work Products (Primary Evidence)
 - Bullets: Jira User Stories with Gherkin criteria; Git Pull Requests with approval hashes; SonarQube quality gate logs; Published Confluence baselines.
 - Box 2: Indirect Work Products (Secondary Evidence)
 - Bullets: Meeting minutes from CCB reviews; Sprint Retrospective action logs; Burnup/Burndown tracking reports; Risk register review logs.
 - Box 3: Affirmations (Stakeholder Interviews)
 - Bullets: Scrum Master interview on impediment tracking; Developer interview on DoD adherence; Product Owner interview on scope management.
5. Draw directional arrows from the three Evidence boxes pointing up into **Tier 3 (Class A Appraisal)**, labeled: Corroboration: Every practice requires Direct + Indirect + Affirmation evidence.
6. Click **File > Export > PNG** for submission.

Activity 5.2: Building an Appraisal Evidence Matrix & Readiness Register in Confluence

- **Tool:** Confluence
- **Objective:** Author an audit-ready SCAMPI Class A Appraisal Evidence Register in Confluence that maps each CMMI Practice Area to live, clickable tool artifacts in Jira, Git, and Confluence.

Step-by-Step Instructions:

1. Open Confluence, navigate to your project space, and click **Create (+)**.
2. Title the page: [Your Name] - CMMI SCAMPI Class A Appraisal Readiness Matrix.
3. Use /heading 2 to add: 1. Objective Evidence Cross-Reference Table.
4. Add text: This document serves as the master evidence navigation ledger for the Lead Appraiser during the upcoming CMMI Level 3 Benchmark Appraisal. It provides direct, verified links to live system artifacts.
5. Type /table and insert a 5-column, 6-row audit ledger:
 - CMMI Practice Area | Specific Practice (SP) | Direct Work Product Link | Indirect Work Product Link | Designated Interviewee
 - Requirements Management (REQM) | SP 1.3: Maintain Bidirectional Traceability | Jira Story #PAY-104 with acceptance criteria | Confluence Traceability Matrix (RTM) | Product Owner
 - Project Planning (PP) | SP 1.2: Establish Estimates of Work Product | Jira Sprint 14 Planning Poker estimates | Sprint Capacity Calculator in Confluence | Modern Project Manager
 - Verification (VER) | SP 2.1: Perform Peer Reviews | Git Pull Request #42 with >=1 approval hash | SonarQube Static Analysis Report Log | Lead Software Engineer
 - Configuration Management (CM) | SP 1.1: Establish Baselines | Git Tag v2.1.0 with SHA-256 hash manifest | Release Deployment Runbook in Confluence | DevOps Lead
 - Risk Management (RSKM) | SP 2.1: Identify & Mitigate Risks | Jira Risk Issue Type #RSKM-01 with flag | 5x5 Risk Matrix & Trigger Log in Miro | Project Manager & Tech Lead
6. Below the table, use /heading 2 to add: 2. Lead Appraiser Interview Defense Strategy.
7. Type /panel and insert an **Info Panel**:
 - Auditor Engagement Rules:
 - 1. Never fabricate paper documentation. Always demonstrate live execution directly in Jira, Confluence, and Git.

- 2. Highlight automated guardrails (e.g., Jira workflow validators) to prove process institutionalization rather than manual memory.
 - 3. Answer the specific practice directly, provide the direct artifact link, and show the matching retrospective or peer review trail.
8. Click **Publish** in the top-right corner.

Post-Release Maintenance and Incident Response: Individual Hands-on Lab Guide

1: Structuring Hotfixes and Emergency Maintenance Releases

Activity 1.1: Designing an Emergency Hotfix & Back-Porting Architecture in Lucidchart

- **Tool:** Lucidchart (or Miro)
- **Objective:** Design an emergency hotfix branching model that isolates production patches directly off release tags, enforces fast-tracked verification stage gates, and synchronizes mainline development branches to satisfy CMMI *Configuration Management (CM)* and *Verification (VER)* standards.

Step-by-Step Instructions:

1. Open Lucidchart, click **+ New > Lucidchart > Blank Document**, and title the document: [Your Name] - Emergency Hotfix & Back-Porting Architecture.
2. From the shape library on the left, drag a **Horizontal Swimlane** container onto the canvas and configure it with **3 lanes**:
 - Label Swimlane 1: Production Mainline (main / release tags)
 - Label Swimlane 2: Emergency Fast-Track Stream (hotfix/* branch)
 - Label Swimlane 3: Active Sprint Development (develop / feature branches)
3. Color-code the lanes: Swimlane 1 dark blue, Swimlane 2 red/orange, and Swimlane 3 light green.
4. In **Swimlane 1 (Production)**, place an initial release tag node:
 - Label: Production Baseline Tag: v2.4.0
5. Model the hotfix branch isolation:
 - In **Swimlane 2 (Hotfix)**, place a process rectangle directly below the v2.4.0 node:
 - Label: Branch Creation: hotfix/v2.4.1 (Branches directly off v2.4.0 tag)
 - Draw a downward directional connector arrow from v2.4.0 in Swimlane 1 to the hotfix branch node in Swimlane 2.
6. In **Swimlane 2**, place 3 sequential process rectangles following the branch node:
 - Block A: Patch Commit: Isolate Fix for Sev 1 Authentication Token Leak
 - Block B: Fast-Track Verification Gate: Targeted Automated Smoke Suite + Security SAST Scan
 - Block C: Emergency CCB Approval Gate: Expedited Sign-off (PM + Lead Architect)
7. Model the production deployment cutover:
 - In **Swimlane 1**, place a deployment node to the right of the original tag:
 - Label: Production Tag: v2.4.1 (SemVer PATCH Deployment)

- Draw an upward arrow from Block C in Swimlane 2 into the new v2.4.1 production node.
- 8. Model the critical back-porting synchronization loop:
 - In **Swimlane 3 (Development)**, place an active sprint commit node labeled develop (Sprint N In-Flight Features).
 - Draw a downward dashed connector arrow from the hotfix/v2.4.1 stream in Swimlane 2 into develop in Swimlane 3.
 - Label this arrow: Mandatory Back-Port Merge: Prevents Defect Regression in Next Scheduled Release.
- 9. Add a callout container at the bottom highlighting CMMI compliance:
 - Text: CMMI CM & VER Audit Rule: Emergency hotfixes must never bypass baseline tagging. Deploying directly to production without back-merging into develop constitutes a severe process non-conformance.
- 10. Click **File > Export > PNG** (or copy the shareable link) for submission.

Activity 1.2: Configuring an Expedited Emergency Hotfix Pipeline and SLA in Jira

- **Tool:** Jira Software
- **Objective:** Configure a dedicated Jira Hotfix issue type, an expedited emergency workflow with strict resolution SLAs, and automated back-port reminders to protect release integrity.

Step-by-Step Instructions:

1. In Jira Software, navigate to **Project settings** (gear icon in the bottom-left sidebar) > **Issue types**.
2. Click **Add issue type**:
 - *Name:* Emergency Hotfix
 - *Description:* Out-of-band production patch triggered by Severity 1 incidents.
 - *Type:* Standard Issue Type
3. Click **Create**.
4. In the left-hand navigation menu, select **Workflows**.
5. Locate the workflow assigned to Emergency Hotfix (or click **Add workflow > Create new**).
6. Build a streamlined 4-step emergency lifecycle:
 - Status 1: Triage & Patching
 - Status 2: Emergency Verification (Staging Smoke Pass)
 - Status 3: Deployed to Production
 - Status 4: Closed & Back-Port Confirmed
7. Click the transition arrow connecting Deployed to Production to Closed & Back-Port Confirmed.
8. In the right-hand properties sidebar, select **Validators** > click **Add validator**.
9. Select **Field Required Validator**, choose Resolution, and enter the error message: CMMI CM RULE: Hotfix cannot close without recording confirmation of back-merge into develop branch.
10. Click **Publish Draft** in the top yellow workflow banner.
11. Navigate to **Project settings > Automation** > click **Create rule**.
12. Set the **Trigger**:
 - Search for Issue transitioned.
 - Set **From status** to Emergency Verification.
 - Set **To status** to Deployed to Production. Click **Save**.
13. Click **+ New action** to select **Create issue**:
 - *Issue Type:* Task

- *Summary:* [MANDATORY BACK-PORT] Merge Hotfix {{issue.key}} into Develop Branch
 - *Description:* Automated Configuration Management action: Ensure code commit for hotfix {{issue.key}} is merged and reconciled against active develop and release staging branches to prevent regression in the next sprint.
 - *Priority:* High
 - Click **Save**.
14. Name the rule: [Your Name] - Hotfix Production Deployment & Back-Port Generator.
 15. Click **Turn on rule**.
 16. Test the workflow: create an Emergency Hotfix ticket, advance it through the stages to Deployed to Production, and confirm that Jira automatically spawns the linked back-port task in your project backlog.

2: Conducting Post-Incident Reviews (PIR) and Root Cause Analysis

Activity 2.1: Conducting a 5-Whys and Fishbone Root-Cause Investigation in Miro

- **Tool:** Miro (or Lucidchart)
- **Objective:** Lead an interactive, blameless Post-Incident Review (PIR) using a 5-Whys chain and Ishikawa (Fishbone) diagram to isolate systemic deficiencies behind a critical production outage to fulfill CMMI *Causal Analysis and Resolution (CAR)* practices.

Step-by-Step Instructions:

1. Open Miro, click + **New board** > **Create a blank board**, and title it: [Your Name] - CMMI CAR Post-Incident Review & RCA Lab.
2. Add a title banner at the top of the canvas using the **Text** tool (T): Post-Incident Review: Outage INC-409 (Gateway 504 Timeout Storm).
3. Draw a large horizontal spine terminating at a red box on the right labeled: Incident Problem: Production API Gateway dropped 65% of checkout traffic for 42 minutes.
4. Draw 4 diagonal rib lines branching off the main spine, categorizing the systemic investigation areas:
 - Rib 1 (Top-Left): Requirements & Edge Cases
 - Rib 2 (Bottom-Left): Architecture & Infrastructure
 - Rib 3 (Top-Right): Testing & Verification Gates
 - Rib 4 (Bottom-Right): Observability & Alerting
5. Attach yellow sticky notes (N) to each rib detailing root-cause contributors:
 - **On Rib 1 (Requirements):** Acceptance criteria did not specify timeout circuit-breaker thresholds when upstream payment partner latency exceeds 2,000ms.
 - **On Rib 2 (Architecture):** Connection pool capacity was locked at a fixed 50 threads; unreturned hanging connections exhausted all available memory.
 - **On Rib 3 (Testing):** Pre-release staging automated suites only tested single-request latency, omitting concurrent connection exhaustion scenarios.
 - **On Rib 4 (Observability):** Alert notifications fired only after 15 consecutive minutes of 5xx errors; on-call team received no progressive degradation warnings.
6. Directly below the Fishbone diagram, create a vertical **5-Whys Analysis Ladder** using 5 interconnected orange sticky notes:

- *Why 1:* Why did the gateway drop checkout traffic? -> Because all server worker threads were locked waiting for upstream responses.
 - *Why 2:* Why did threads remain locked? -> Because the third-party partner experienced network latency and requests lacked client-side timeouts.
 - *Why 3:* Why were there no timeouts configured? -> Because default connection settings were pushed during an unreviewed configuration change.
 - *Why 4:* Why was the configuration change unreviewed? -> Because environment variable modifications were classified as "minor" and bypassed peer review.
 - *Why 5 (Systemic Root Cause):* -> Process Gap: Our Definition of Done and CI/CD pipeline did not treat infrastructure/environment configurations with the same peer-review and testing rigor as source code.
7. To the right of the 5-Whys ladder, create a green container titled: CMMI CAR Institutional Remediation Actions.
 8. Add 2 green sticky notes inside:
 - *Action 1:* Update PAL-STD-005 (DoD): Require peer review sign-offs and automated lint validation for all Kubernetes Helm and environment config files.
 - *Action 2:* CI/CD Gate: Deploy automated Chaos-Mesh latency injection tests in Staging prior to release candidate sign-off.
 9. Draw a solid line from the 5th "Why" sticky note directly into the CAR Action container to demonstrate causal closure.
 10. Click **Export this board** > **Save as image (PNG)** for submission.

Activity 2.2: Authoring a Formal Post-Incident Review (PIR) Dossier in Confluence

- **Tool:** Confluence
- **Objective:** Author an executive-level, blameless Post-Incident Review (PIR) report in Confluence that documents incident chronology, user impact telemetry, systemic root causes, and CMMI *Causal Analysis and Resolution (CAR)* preventive commitments.

Step-by-Step Instructions:

1. Open Confluence, navigate to your assigned project space, and click **Create (+)**.
2. Title the page: [Your Name] - Post-Incident Review (PIR): Production Outage INC-409.
3. Use /heading 2 to add: 1. Incident Summary & Executive Blast Radius.
4. Type /table and insert a 2-column, 6-row metadata table:
 - Incident Parameter | Operational Measurement
 - Incident ID & Severity | INC-409 | Severity 1 (Critical Outage)
 - Date & Duration | October 22, 2026 | 14:12 - 14:54 UTC (42 minutes total downtime)
 - Customer / Business Impact | 1,420 transaction attempts aborted; estimated \$118,000 lost checkout volume.
 - Mean Time to Detect (MTTD) | 16 Minutes (Observability Alert fired at 14:28 UTC)
 - Mean Time to Resolve (MTTR) | 26 Minutes (Traffic successfully recovered via rollback to v2.4.0)
 - Incident Commander | [Your Name], Modern Project Manager
5. Use /heading 2 to add: 2. Incident Chronology & Event Timeline.
6. Type /table and insert a 3-column, 5-row event timeline table:
 - Timestamp (UTC) | System State & Operational Event | Actor / Trigger

- 14:00 | Release candidate v2.4.1 deployed to 100% production traffic. | Scheduled Deployment Pipeline
 - 14:12 | Third-party bank API latency climbs from 120ms to 4,500ms. | External Upstream Outage
 - 14:18 | Gateway thread pool hits 100% saturation; HTTP 504 errors surface. | Production System Telemetry
 - 14:28 | On-call Incident Commander notified via MS Teams war-room alert. | Datadog Automated Alert
 - 14:54 | Rollback to baseline v2.4.0 completed; 100% transaction traffic restored. | DevOps Lead & Incident Commander
7. Use /heading 2 to add: 3. CMMI CAR Root Cause & Systemic Findings.
 8. Type /panel and insert an **Info Panel** detailing systemic findings:
 - Blameless Cultural Finding:
 - The outage was not caused by operator error or individual oversight. The failure was induced by an absence of automated circuit-breaker timeout enforcement in our application templates and an alert notification threshold that allowed silent thread exhaustion for 16 minutes.
 9. Use /heading 2 to add: 4. Preventive Actions & PAL Updates (CMMI CAR SP 2.1).
 10. Type /table and insert a 5-column, 3-row remediation tracking matrix:
 - Action Item ID | Preventive Engineering Task Description | Assigned Owner | Target Completion Date | Jira Ticket Key
 - CAR-409-01 | Implement Resilience4j circuit-breaker with a hard 1,500ms fallback timeout on all external API clients. | Lead Backend Engineer | Next Sprint (Sprint 15) | PAY-340
 - CAR-409-02 | Recalibrate Datadog alert thresholds: alert on-call teams if HTTP 5xx error rates exceed 1.0% for >2 minutes. | Lead DevOps / SRE | Sprint 15 | OPS-112
 11. Below the matrix, insert a **Success Panel** (/panel):
 - Heading: Organizational Learning & Closure Attestation:
 - Text: Preventive tasks CAR-409-01 and CAR-409-02 have been injected into the active Jira sprint backlog as mandatory technical debt refactoring items. This PIR dossier satisfies CMMI Level 3 CAR audit compliance.
 12. Click **Publish** in the top-right corner.